STUDY GUIDE

.NET Tech Lead

Auth, Scalability, Azure, and SQL

Deep questions and answers for an 8-year full-stack .NET developer.

Authentication and authorization · JWT · Web API scale · Azure · SQL Server

Companion to the 10-day roadmap and the deep dive. Those guides are the schedule and the first worked example. This guide is the follow-up a panel asks after the 90-second answer.

Prepared 31 October 2026. Examples target C# on .NET 10 LTS, supported through 14 November 2028. .NET 8 and .NET 9 reach end of support in November 2026.

How to use this companion

Read a section until you can redraw the example. Then close it and answer the question out loud in under two minutes. Use the lead line as the last sentence. If the follow-up is a trap from a warn box, say the trap and the rule you would put in review. Swap in a system you have operated when you have one. Until then, stay on the Orders API and label every number that is an assumption.

1. Speak the definition in one sentence.
2. Attach it to POST /orders, the token, Azure SQL, or the slot.
3. Name the failure: who feels it, and what you roll back.
4. Stop. A third feature is how a clear answer becomes a tour.

Section	You will be able to teach
Authentication and authorization	Schemes, policies, resource checks, tenant isolation, 401 and 403.
JWT	What a token proves, what it leaves readable, and the validation order.
Web API scalability	Stateless instances, async, cache, backpressure, and the 50 request assumption.
Azure	The host, the identity, the queue, the slot, and the rollback.
SQL	A plan, an index, a tenant-safe write, and six queries you can defend.

The running example

A partner POSTs an order and may retry. A customer GETs the order. The API runs on Azure App Service. Azure SQL holds the order, the lines, the customer, and the idempotency key. The email worker is an Azure Function on the isolated worker model, triggered by Azure Service Bus. The caller's token comes from Microsoft Entra ID. The process reaches Azure SQL, Key Vault, and Service Bus with a managed identity.

Scale you may assume out loud, and you label it as an assumption: about 50 requests a second at peak, about 2 million orders a year, p95 under 300 milliseconds for the read, email within a minute. A first-party Angular app talks to a BFF when you introduce one. The partner calls the API with a bearer token. One process does not have to serve both surfaces.

What this guide will not do

It will not invent an Azure price, a SKU quota, or a latency you should memorize. When a limit matters, name the document or the metric you would open. It will not claim these SQL examples were executed in a session you did not run. Confirm a plan on your own server before you argue row counts.

Authentication and authorization

Authentication answers who called. Authorization answers whether that caller may do this. On the Orders API those are two steps in the pipeline, in that order. Authentication builds a ClaimsPrincipal, or it fails. Authorization reads that principal, the endpoint policy, and sometimes the order row. A missing token and a token that lacks orders.write are different outcomes, and the client behaves differently for each.

The partner who POSTs an order presents a bearer token issued by Entra ID. A first-party Angular app, when you add one, talks to a BFF. The browser holds an HttpOnly cookie. The BFF holds the tokens. The Orders API still validates a bearer token on the call the BFF or the partner makes. Cookie authentication and bearer authentication solve two surfaces. One deployment does not have to enable both.

Handlers, schemes, and the default scheme

An authentication scheme is a name plus a handler type. The handler implements AuthenticateAsync, ChallengeAsync, and ForbidAsync. AuthenticateAsync inspects the request and returns a result: success with an AuthenticationTicket, failure when a credential was present and bad, or no result when this handler has nothing to read. The ticket carries the ClaimsPrincipal.

AddAuthentication registers the services and the default scheme. The default scheme is the one the rest of the pipeline uses when a call does not name a scheme. You can split that into a default authenticate scheme, a default challenge scheme, and a default forbid scheme when they differ. A BFF often authenticates with a cookie and challenges with the OpenID Connect handler. A pure Orders API sets a single default: the JWT bearer scheme.

UseAuthentication runs the default handler and assigns HttpContext.User. UseAuthorization runs after it. Authorization that runs before authentication sees an anonymous user. The endpoint then treats a missing token as a forbid, and the client shows access denied where it should have shown a sign-in.

```
app.UseAuthentication();
app.UseAuthorization();

builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options =>
    {
        options.Authority = builder.Configuration["Entra:Authority"];
        options.Audience = builder.Configuration["Entra:Audience"];
        options.MapInboundClaims = false;
    });
```

Authority is the Entra metadata address, typically the v2.0 issuer for the directory that issues your tokens. The handler loads the signing keys from that document over HTTPS. Audience is the API's application ID URI, the value in the aud claim the partner requested. The full TokenValidationParameters sketch is in the JWT section. Leave it on. The four validation switches stay enabled.

MapInboundClaims left at its historical default rewrites short claim names into long URI claim types. A policy that looks for scp or roles then fails on a token you already proved was signed. This guide sets MapInboundClaims to false and authorizes on the names Entra issued.

Challenge and Forbid

Challenge means the request has no usable identity. Forbid means the identity is known and the policy refused it. The bearer handler writes 401 and a WWW-Authenticate header for a challenge. It writes 403 for a forbid. The cookie handler's challenge redirects to the login path. That redirect is correct for a browser on the BFF. It is a bug on a partner API: the partner receives 302 and a login HTML page.

The authorization middleware chooses for you on an [Authorize] endpoint. An anonymous user, or a handler failure such as a bad signature, becomes a challenge. An authenticated user who fails the policy becomes a forbid. Results.Challenge and Results.Forbid are the same pair when you decide inside a handler after you have loaded a resource. Match the scheme to the surface so challenge means the status code that surface expects.

Situation	Pipeline result	Partner API status
No Authorization header	Challenge	401
Token present, signature, issuer, audience, or lifetime fails	Challenge	401
Token valid, scope or policy fails	Forbid	403
Token valid, order id is outside the tenant	Your action returns NotFound	404
Browser on the BFF, no session	Cookie challenge	Redirect to login

404 for a cross-tenant id is a product choice. The query includes the tenant, so the row is absent in this caller's scope. The response does not confirm that some other tenant owns that id. Document it. A missing orders.write scope is still 403, because the middleware refused the route before a row was involved. A fresh token with the same claims fails the same way. 401 is the signal to sign in again.

Cookie, bearer, and the grant

A pure API validates Authorization: Bearer. The partner is a confidential client. It uses the client credentials grant when it acts as itself, and the token carries app roles. A human user uses the authorization code grant with PKCE. The client sends a code_challenge, method S256, on the authorize redirect. The token endpoint receives the code and the original code_verifier. Entra returns the access token and, when you asked for offline_access, a refresh token.

The BFF is the confidential client for the Angular app. It redeems the code. It stores the access token and the refresh token on the server. It sets a cookie that is HttpOnly, Secure, and SameSite. The browser sends that cookie to the BFF. Script on the page cannot read it. The BFF attaches the access token when it calls the Orders API.

```
builder.Services.AddAuthentication(CookieAuthenticationDefaults.AuthenticationScheme)
    .AddCookie(options =>
    {
```

```
        options.Cookie.HttpOnly = true;
        options.Cookie.SecurePolicy = CookieSecurePolicy.Always;
        options.Cookie.SameSite = SameSiteMode.Lax;
        options.LoginPath = "/bff/login";
    });
```

A cookie is attached by the browser. A bearer header is attached by code that chose to attach it. The BFF therefore protects cookie-authenticated POSTs against cross-site request forgery: SameSite, and an antiforgery token on unsafe methods. The partner API does not take a session cookie, so it does not add a browser antiforgery token. It demands the bearer token instead.

A public browser app can run authorization code with PKCE and redeem the code in the page. This system's choice is still the BFF, so the refresh token stays on the server. Two schemes in one process are a later option: cookie for the BFF routes, bearer for /partner. A policy scheme can forward to the handler whose credential is present. Build that when one host truly serves both. Ship the partner API on bearer first.

Interview trap

Implicit flow, response_type=token, puts the access token in the URL fragment. Leave it out of the design. An access token in localStorage or sessionStorage is readable by any script that runs on the origin. A long-lived token in the browser is a stolen credential with a wide window. The OAuth security guidance for a browser client is authorization code with PKCE, and this guide keeps those tokens on the BFF.

Policies, requirements, and handlers

A role is a claim. A policy is a named rule in the composition root. [Authorize(Roles = "Admin,Support")] scatters an OR-list of role strings across actions. Renaming a role means a sweep of controllers. A policy named orders.write is one registration, one test, and one attribute. The endpoint stays stable when the rule grows from a role to a scope plus a tenant claim.

IAuthorizationRequirement is a marker. A handler is AuthorizationHandler of that requirement. Requirements on one policy are combined: every requirement must succeed. Several handlers for one requirement are alternate ways to succeed: the first success satisfies that requirement. context.Succeed records success. Returning without Succeed is an ordinary failure. context.Fail is an explicit deny that stops the remaining handlers. Use Fail when you mean deny even if another handler would have allowed the call.

Entra puts delegated permissions in one scp claim, space-separated. It puts app roles in one or more roles claims, each value a single role. RequireClaim("roles", "orders.write") matches an app role. RequireClaim("scp", "orders.read") looks for a claim whose entire value is orders.read, so a token whose scp is "orders.read orders.write" fails it. Split scp in a handler.

```
public sealed class ScopeRequirement : IAuthorizationRequirement
{
    public ScopeRequirement(string scope) => Scope = scope;
    public string Scope { get; }
}

public sealed class ScopeHandler : AuthorizationHandler<ScopeRequirement>
```

```
{
    protected override Task HandleRequirementAsync(
        AuthorizationHandlerContext context,
        ScopeRequirement requirement)
    {
        foreach (var claim in context.User.FindAll("scp"))
        {
            foreach (var scope in claim.Value.Split(' ', StringSplitOptions.RemoveEmptyEntries))
            {
                if (string.Equals(scope, requirement.Scope, StringComparison.Ordinal))
                {
                    context.Succeed(requirement);
                    return Task.CompletedTask;
                }
            }
        }
        return Task.CompletedTask;
    }
}

builder.Services.AddSingleton<IAuthorizationHandler, ScopeHandler>();
builder.Services.AddAuthorizationBuilder()
    .SetFallbackPolicy(new AuthorizationPolicyBuilder()
        .RequireAuthenticatedUser()
        .Build())
    .SetDefaultPolicy(new AuthorizationPolicyBuilder()
        .RequireAuthenticatedUser()
        .Build())
    .AddPolicy("orders.read", policy =>
    {
        policy.RequireAuthenticatedUser();
        policy.AddRequirements(new ScopeRequirement("orders.read"));
    })
    .AddPolicy("orders.write", policy =>
    {
        policy.RequireAuthenticatedUser();
        policy.AddRequirements(new ScopeRequirement("orders.write"));
        policy.RequireClaim("tenant_id");
    });
```

The same registration exists as services.AddAuthorization(o => o.AddPolicy(...)). AddAuthorizationBuilder is the fluent form. Handlers that do not take scoped services are singletons. A handler that opens a DbContext is scoped, or it creates a scope inside the call.

The default policy is what a bare [Authorize] applies. Here it requires an authenticated user. The fallback policy applies to endpoints that carry no authorization metadata. Here it requires an authenticated user, so a new minimal API you forgot to mark is still closed. AllowAnonymous is how you opt out. The liveness and readiness probes are anonymous because the platform has no user token. The probe body stays boring: a status, no order data.

Partner app tokens often have roles and no scp. Add a second handler for the same requirement that succeeds when roles contains orders.write, or register a partner policy that uses RequireClaim("roles",

"orders.write"). Say which token shape each route accepts. A route that silently accepts either shape is a decision you write down.

```
[Fact]
public async Task Write_scope_succeeds_when_scp_lists_several_values()
{
    var user = new ClaimsPrincipal(new ClaimsIdentity(
    [
        new Claim("scp", "orders.read orders.write"),
        new Claim("tenant_id", "10")
    ], authenticationType: "Bearer"));
    var context = new AuthorizationHandlerContext(
        [new ScopeRequirement("orders.write")], user, resource: null);
    await new ScopeHandler().HandleAsync(context);
    Assert.True(context.HasSucceeded);
}
```

Resource-based access to an order

The endpoint policy is the coarse gate: scope, and a tenant claim that parses. It runs before the action, so it cannot see the order. Resource-based authorization runs after you have the entity. You pass the order to IAuthorizationService.AuthorizeAsync. The handler receives the requirement and the resource.

Load the order with the tenant predicate already applied. The handler is the second check, for a rule the predicate does not express. In this example a partner token may also carry customer_id. That partner may read only that customer's order. A staff token inside the tenant omits customer_id and may read any order the tenant query returned.

```
public sealed class OrderAccessRequirement : IAuthorizationRequirement
{
}

public sealed class OrderAccessHandler
    : AuthorizationHandler<OrderAccessRequirement, Order>
{
    protected override Task HandleRequirementAsync(
        AuthorizationHandlerContext context,
        OrderAccessRequirement requirement,
        Order resource)
    {
        var tenantClaim = context.User.FindFirst("tenant_id")?.Value;
        if (!int.TryParse(tenantClaim, out var tenantId) || tenantId != resource.TenantId)
            return Task.CompletedTask;

        var customerClaim = context.User.FindFirst("customer_id")?.Value;
        if (customerClaim is not null &&
            (!int.TryParse(customerClaim, out var customerId) || customerId !=
resource.CustomerId))
            return Task.CompletedTask;

        context.Succeed(requirement);
        return Task.CompletedTask;
```

```
    }
}

app.MapGet("/orders/{id:long}", async (
    long id,
    ClaimsPrincipal user,
    AppDbContext db,
    IAuthorizationService auth,
    CancellationToken ct) =>
{
    var tenantId = int.Parse(user.FindFirstValue("tenant_id")!);
    var order = await db.Orders.AsNoTracking()
        .SingleOrDefaultAsync(o => o.Id == id && o.TenantId == tenantId, ct);
    if (order is null)
        return Results.NotFound();

    var decision = await auth.AuthorizeAsync(user, order, "order.access");
    if (!decision.Succeeded)
        return Results.Forbid();

    return Results.Ok(OrderDto.From(order));
}).RequireAuthorization("orders.read");
```

Register order.access with AddRequirements(new OrderAccessRequirement()), and register the handler. A global query filter on TenantId is a safety net for ordinary LINQ. IgnoreQueryFilters and a raw SQL path skip it. The test that matters sends tenant A's token and tenant B's order id, and expects 404.

Endpoint metadata

[Authorize(Policy = "orders.write")] on a controller action, and RequireAuthorization("orders.write") on a minimal API, both add authorization metadata. [AllowAnonymous] and AllowAnonymous() add the opt-out. The middleware reads that metadata. It does not guess from a comment or from a folder name.

Calling RequireAuthorization twice combines the policies. The endpoint must satisfy each one. A group that requires orders.read plus a POST that requires orders.write demands both. Put one policy on the route unless you mean the combination. A bare [Authorize] uses the default policy. With the fallback above, a route with neither attribute still requires a user.

```
app.MapPost("/orders", CreateOrder).RequireAuthorization("orders.write");
app.MapGet("/orders/{id:long}", GetOrder).RequireAuthorization("orders.read");
app.MapHealthChecks("/health/live").AllowAnonymous();
app.MapHealthChecks("/health/ready").AllowAnonymous();
```

Tenant isolation and object-level access

Broken object level authorization, often called IDOR in older write-ups, is a guessable id and a query that loads by that id alone. The caller changes /orders/10 to /orders/11 and receives another tenant's order. The fix is a predicate: id and tenant together. The tenant value comes from the authenticated identity.

Entra's tid claim is the directory id. Use it as the business tenant when one customer is one directory. When many customers share a directory, the business tenant is a custom claim this guide calls tenant_id, issued by your claims mapping, or it is a row you resolve from the client application id in azp or appid. That lookup is server-side. The JSON body is the wrong source in every one of those shapes. A field named TenantId on the create request is over-posting. Reject it in validation.

The Day 3 sketch reads a claim it called tid because that sample token was shaped that way. In the room, say which claim is the directory and which claim is the customer. Then say the body cannot override either.

```
int tenantId = user.FindFirst("tenant_id") is { Value: { } raw } && int.TryParse(raw, out var id)
    ? id
    : await tenants.RequireByClientIdAsync(user.FindFirstValue("azp")!, ct);

if (body.TenantId is not null)
    return Results.ValidationProblem(new Dictionary<string, string[]>
    {
        ["tenantId"] = ["TenantId is not accepted from the body."]
    });
```

Interview trap

A valid signature with the wrong audience accepts another API's token. A filter that hides rows in EF is not the boundary once someone calls IgnoreQueryFilters or FromSqlRaw. Returning 403 for a missing token sends the partner into an access-denied path. Returning the other tenant's order with 200 is the incident.

Where the secrets live

An Entra-signed token is validated with public keys from the metadata document. You do not put Entra's private key in the API, and you do not commit a copy of a signing key to stand in for metadata. An API that issues its own tokens loads the signing material from Key Vault through configuration at startup. The value is absent from appsettings.json in git.

Downstream calls use the process identity. The App Service managed identity is the Azure SQL user, the Key Vault reader, and the Service Bus sender. The user's access token is the authorization decision for the human or the partner. It is a different identity from the login that opens SQL. Forwarding the raw user token to every internal hop is a choice you make when that hop must see the user. The Orders API's own data access uses the managed identity.

RS256 is the usual shape for an IdP-signed access token: the issuer signs with a private key, and every API verifies with the published public key. A symmetric key, HMAC such as HS256, fits a small API that issues tokens to callers you control. The secret lives in Key Vault. Any service that holds that secret can mint tokens that the others will trust. A shared symmetric secret across many services is a poor boundary. Prefer the IdP and asymmetric verification when more than one service must accept the token.

Questions and answers

Q1. What is the difference between authentication and authorization?

Answer. Authentication builds the ClaimsPrincipal: who called, from a bearer token or a BFF cookie. Authorization decides whether that principal may call this action or touch this order. On POST /orders the bearer handler validates the Entra token first. The orders.write policy runs next. The action then binds the tenant from the token and writes the order. A failure in the first step is 401 on the partner API. A failure in the second step is 403. A row that is outside the tenant is 404 because the query never sees it.

> **Say this as a tech lead**
>
> I keep the words apart in review. A PR that checks a role before it has a principal, or that treats 403 as 'please log in', has mixed them up.

Q2. What is a scheme, a handler, and the default scheme?

Answer. A scheme is a name bound to a handler. The handler reads a credential and returns a ticket, a failure, or no result. AddAuthentication(JwtBearerDefaults.AuthenticationScheme) makes bearer the default for authenticate, challenge, and forbid on the partner API. A BFF can authenticate with a cookie and challenge with OpenID Connect, because those defaults are separate properties when you need them to differ. UseAuthentication assigns HttpContext.User. UseAuthorization reads it. The JWT section shows the handler options for issuer, audience, lifetime, and signature.

> **Say this as a tech lead**
>
> I name the default scheme in the first minute. A partner API on the cookie scheme redirects to a login page, and the partner treats that as a broken contract.

Q3. When do you Challenge, and when do you Forbid?

Answer. Challenge when there is no usable credential: missing header, bad signature, wrong issuer, wrong audience, expired token. The bearer handler turns that into 401. Forbid when the principal is authenticated and the policy failed: the scope is absent, the tenant claim is absent, the resource handler did not succeed. That is 403. The cookie handler's challenge is a redirect, which belongs on the BFF. Inside a minimal API, Results.Challenge and Results.Forbid call the same handlers. The middleware already did this for [Authorize] before your delegate ran. You call them yourself for an imperative resource check.

> **Say this as a tech lead**
>
> 401 means the client should obtain a new token. 403 means a new token with the same claims will fail until someone grants access. I do not collapse those for a simpler client.

Q4. How do you choose cookie authentication versus JWT bearer?

Answer. The partner Orders API uses JWT bearer. The caller sends Authorization: Bearer, and the API validates it on every request. The first-party Angular app uses a BFF with a cookie session: HttpOnly, Secure, SameSite. The BFF stores the access token and the refresh token, and it attaches the access token when it calls the API. The browser never receives the access token. If one process later serves browser routes and

partner routes, cookie and bearer are two schemes, forwarded by which credential is present. I ship them as two surfaces first.

Say this as a tech lead

I can draw the browser, the BFF, and the partner as three boxes. The access token sits in the BFF or in the partner's secret store. It does not sit in the page.

Q5. Which OAuth grant do you use for the interactive user, and where do the tokens live?

Answer. Authorization code with PKCE, code_challenge_method S256. The user signs in at Entra. The BFF redeems the code with the code_verifier and a confidential client. The access token and the refresh token stay on the server. The browser gets the session cookie. offline_access is how you ask Entra for a refresh token. The partner application, when it has no user, uses client credentials and an app role. I keep implicit flow out of the design. I keep access tokens out of localStorage and sessionStorage.

Say this as a tech lead

PKCE protects the code on the way back. The BFF protects the tokens after that. XSS that reads the page still cannot read an HttpOnly cookie.

Q6. What do you configure on AddJwtBearer, and what do you refuse to turn off?

Answer. Authority and audience, MapInboundClaims false, HTTPS metadata left on. Issuer, audience, lifetime, and signing key validation stay enabled. Clock skew is the library default of five minutes unless I set a smaller window on purpose and say why. I pin the algorithm to RS256 for Entra access tokens this API accepts, after I have seen a real token in the test directory. I do not install a SignatureValidator that returns success. I do not accept alg none. The self-issued sketch, when I am the issuer, loads a symmetric key from configuration that Key Vault filled, and it pins HMAC. That key is a different scheme from the Entra scheme.

Say this as a tech lead

If a teammate disables issuer or signature validation to unblock a demo, the PR does not merge. The demo gets a correct authority instead.

Q7. Why is a policy a better gate than [Authorize(Roles = "...")] on each action?

Answer. The role string is a claim, and it is a blunt OR. Scattered role attributes drift: one action says Admin, the next says Admin,Support, the next forgot the attribute. orders.write lives in one AddPolicy call. The attribute only names it. The rule can require a scope, a tenant claim, and a handler, and a unit test can pass a principal through that handler. When the business word changes from a role to a scope, the endpoints stay. I still use a role when the token truly carries app roles. I reference it inside the policy.

Say this as a tech lead

In review I look for a new route with no policy name. The fallback policy is the backstop. The named policy is the permission we meant.

Q8. What is the difference between the fallback policy and the default policy?

Answer. The default policy runs when the endpoint says [Authorize] or RequireAuthorization() with no name. The fallback policy runs when the endpoint has no authorization metadata at all. On this API both require an authenticated user. Named policies add the scope. Health checks are AllowAnonymous so the probe still works. Without a fallback, a new MapGet that nobody decorated is public. Without a thought-out default, a bare [Authorize] might mean something the author did not read.

> **Say this as a tech lead**
>
> I would rather a forgotten endpoint return 401 than accept an anonymous POST. Public routes are an explicit AllowAnonymous I can see in the diff.

Q9. How do you implement resource-based authorization for an order?

Answer. The route policy orders.read runs first and checks the scope. The action loads the order with Where Id and TenantId, and returns 404 when that row is absent. It then calls AuthorizeAsync with the order and the order.access policy. The handler succeeds when tenant_id matches the row and, if customer_id is present, that matches too. Staff tokens omit customer_id. I register a handler of AuthorizationHandler<OrderAccessRequirement, Order>. The requirement has no place on the attribute alone, because the attribute runs before the order exists.

> **Say this as a tech lead**
>
> Coarse policy on the route, tenant predicate in the query, resource handler for the extra rule. I can point at a test for each.

Q10. Where does the tenant id come from, and what is BOLA?

Answer. BOLA is access to an object by guessing or changing its id, when the server checks the id and skips the owner. The Orders query is Id plus tenant. The tenant is tenant_id on the token when many customers share one Entra directory. It is Entra's tid when one customer is one directory. It is a server-side map from azp when the partner token identifies an application and the claim is not on the token. It is never a property of the JSON body. A global query filter repeats the rule for ordinary LINQ. A test with tenant A's token and tenant B's id is the control.

> **Say this as a tech lead**
>
> I reject TenantId on the create model in validation. The day someone binds the entity directly, that rejection is what saves us.

Q11. What do you mark [AllowAnonymous], and what does minimal API metadata change?

Answer. The live and ready probes are anonymous. Anything else on the Orders API needs a named policy. [Authorize] and RequireAuthorization write metadata the middleware reads. AllowAnonymous writes the opt-out, including an opt-out from the fallback policy. Two RequireAuthorization calls both apply. I put a single policy on each route so the scope is obvious in the endpoint list. Controller filters and minimal API metadata are two ways to attach the same decision.

Say this as a tech lead

I can open the endpoint list and read the policy next to the verb. A route I cannot find in that list is the one I am worried about.

Q12. When would one app use more than one authentication scheme?

Answer. When one process really has two surfaces. The BFF routes authenticate the cookie. The partner routes authenticate bearer. A forwarding scheme picks the handler from the request: an Authorization bearer header selects JWT, otherwise the cookie. Challenge and forbid must follow the same choice, or a browser receives 401 and a partner receives a redirect. I do not add the second scheme on the first day of the partner API. I add it when the BFF moves into this host for a reason I can say.

Say this as a tech lead

Two schemes are a host decision. They are not a requirement of Entra, and they are not a requirement of having both a web app and an API in the company.

Q13. Where do signing keys live, and when is RS256 the right story versus a symmetric key?

Answer. Entra tokens: the API trusts the metadata document and verifies with the published public key. RS256 is the pin for those tokens once I have confirmed the alg on a real access token. Tokens I issue myself: the symmetric secret or the private key is loaded from Key Vault, and the appsettings file in git does not contain it. HMAC is enough for a local issuer with a small set of callers. The moment several services share that secret, any one of them can mint tokens for the others. That is a poor boundary. Those services should validate an IdP-signed token instead. The managed identity, not the user token, is what opens Azure SQL.

Say this as a tech lead

I can name three identities: the caller in the token, the app's managed identity, and the pipeline identity that runs DDL. The signing key is a fourth secret, and it stays in the vault.

Q14. What is clock skew, and what do you set it to?

Answer. Clock skew is the allowance for exp and nbf when the issuer's clock and the API's clock disagree. The library default is five minutes. I leave it there, or I set a smaller value when I know the hosts sync and I want a tighter expiry. A large skew extends every expired token by that much. A skew of zero fails tokens when the machines are a few seconds apart, which shows up as random 401s after deploy. I do not use skew to paper over a token that is expired by hours. That token needs a refresh.

Say this as a tech lead

Skew is a clock tolerance. It is not a session length. The access token lifetime is the session length I chose with the identity team.

JWT

The partner token on POST /orders is a signed JSON Web Token. You can read it in a debugger by splitting on the dots and base64url-decoding the first two parts. You still refuse it until the handler has checked the signature, the issuer, the audience, and the lifetime. Claims are an authorization input after that, not before.

Header, payload, and signature

A signed token in compact form has three parts: header, payload, signature. The header names the algorithm and often a kid, the key id. The payload is the claims set: iss, aud, exp, sub, and the application claims. The signature covers the header and the payload. The handler selects a key you already trust, checks that the algorithm is one you allow, and verifies the signature.

That token is a JWS, a signed token. The payload is encoded, and it is not encrypted. Anyone who holds the token can decode the claims: the partner, the browser if you leaked it there, a log line, a trace tool. An encrypted token is a JWE. A JWE uses a different compact form, five parts, and the payload is ciphertext. Entra access tokens that this API validates are signed tokens. Put an identifier in a claim. Put a secret somewhere else. If a claim would hurt you when a partner reads it back, it does not belong in the access token.

```
header.payload.signature

header   {"alg":"RS256","typ":"JWT","kid":"<key id from the issuer>"}
payload  {"iss":"https://login.microsoftonline.com/<directory>/v2.0",
          "aud":"api://orders","exp":...,"scp":"orders.write",
          "oid":"<object id>","tenant_id":"10"}
signature  RS256 over the first two segments
```

Access token and refresh token

The access token is what the Orders API sees on each call. Keep it short. You read the real lifetime in the Entra token lifetime policy and in the exp claim of a token from the test tenant. A stolen access token works until exp, because the API validates it locally and does not call Entra on every request.

The refresh token is a different credential. The client sends it to the token endpoint, not to the Orders API. The token endpoint returns a new access token. Rotation means each refresh also returns a new refresh token and the previous one is dead. Store a hash of the current refresh token with the session, in the BFF or in the partner's confidential store. If a retired refresh token appears, treat the family as stolen and revoke it. The partner who can store a secret keeps the refresh token in that secret store. The browser keeps neither.

A long-lived access token is the wrong fix for a client that is tired of refreshing. You cannot take it back before exp without a denylist you then have to check on every request. It gets copied into logs and into support screenshots. It widens the window after XSS or a leaked proxy log. Short access tokens plus refresh rotation keep that window small. Revocation becomes 'refuse the refresh'.

Interview trap

Sending the refresh token to the Orders API, logging either token, or putting the access token in a query string. The query string lands in access logs. A refresh token that never rotates still works after the user thought they signed out, until its own long lifetime ends.

Claims you actually authorize on

iss, aud, and exp are validation, and they happen before business rules. sub is the subject. On Entra, sub is pairwise per application, so it changes if you read it from another app. oid is the object id of the user or the service principal in the directory, and it is the stable id to store. tid is the directory. tenant_id in these examples is the business tenant when many customers share a directory. scp is delegated scope, space-separated. roles is the app role, one claim value per role. azp or appid is the client application. Use that as the partner identity when you map the caller to a tenant row, and as a rate-limit key. Authorize on a claim the issuer put in the token. Ignore a copy of the same name in the JSON body.

Claim	What you use it for
iss	The token came from the directory you trust.
aud	The token was requested for this API.
exp and nbf	The token is inside its lifetime, plus clock skew.
oid	The user or service principal id you persist.
sub	The subject. On Entra, prefer oid when you need a stable user id.
tid	The Entra directory. The customer id only when one customer is one directory.
tenant_id	The business tenant in a shared directory. Custom claim, or map it from azp.
scp	Delegated permission. Split on spaces before you compare.
roles	App role for a client-credentials partner.
azp / appid	Which application called. Map to a tenant. Partition a rate limit.

Validation order

This is the order you require. The handler finishes it before the action runs. You do not branch on tenant_id from a token whose signature you have not checked.

1. The compact token has three segments. A five-segment JWE is a different parser, and this API is not that parser.
2. The header alg is on your allow-list. alg none fails. An alg you did not configure fails.
3. kid selects a key you already hold from the issuer metadata, or you use the one key you configured. The token does not supply the key material.
4. The signature verifies with that key. Failure is 401.
5. iss matches the authority you configured.
6. aud matches this API.

7. nbf and exp pass, with the clock skew you set. Failure is 401.
8. Then the policy reads scp, roles, and tenant_id. Failure of the policy is 403.

A wrong audience and a bad signature are both 401. The caller obtained a token for something else, or the token was damaged. A right token with the wrong scope is 403. Log the failure reason on the server. Keep the response body free of key material and free of a stack trace.

Algorithm confusion

The header is attacker-controlled until the signature check. alg none with an empty signature is a token some naive libraries once accepted. Pin the algorithm. A second failure is letting the token decide the key: a kid that makes you download a URL from the header, or a header that switches you from RS256 to HS256.

The classic confusion is an API that accepts both RSA and HMAC on the same validation parameters. The attacker signs with HMAC, using the API's public key bytes as the HMAC secret. The API verifies HS256 with those same bytes and accepts a token the attacker wrote. The Entra scheme on this API allows RS256 and takes keys only from the metadata document. A local issuer scheme allows HMAC and takes the secret only from Key Vault. Those are two schemes. They do not share one bag of keys and one bag of algorithms.

Interview trap

Disabling signature validation, accepting alg none, or trusting a key embedded in the token header. Also a test helper that stays registered in production and treats every token as valid. The test signs a token with a test key, or it uses the test directory. It does not delete the check.

Reference tokens

A JWT carries its claims, and the API checks them locally. Revocation waits for exp unless you add a denylist. A reference token, sometimes called an opaque token, is a handle. The API introspects it at the issuer, or looks it up in a store the issuer shares. Revocation is visible on the next introspection. The cost is a network call or a cache of introspection results, and that cache has its own lifetime. For the Orders API, Entra JWT validation is the default. Mention a reference token when the panel asks how you would revoke a token in seconds, and say what you would cache and for how long.

A JwtBearer configuration you can defend

The first sketch is the partner API. Authority fills issuer and signing keys from metadata. You set the audience. You keep the validation flags on. You pin RS256 after you have confirmed it on a token from your tenant. Clock skew of two minutes is a team choice, tighter than the five-minute library default, and you say it is a choice.

```
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options =>
    {
        options.Authority = builder.Configuration["Entra:Authority"];
        options.Audience = builder.Configuration["Entra:Audience"];
        options.MapInboundClaims = false;
```

```
        options.TokenValidationParameters.ValidateIssuer = true;
        options.TokenValidationParameters.ValidateAudience = true;
        options.TokenValidationParameters.ValidateLifetime = true;
        options.TokenValidationParameters.ValidateIssuerSigningKey = true;
        options.TokenValidationParameters.ValidAlgorithms = new[] { SecurityAlgorithms.RsaSha256 };
        options.TokenValidationParameters.ClockSkew = TimeSpan.FromMinutes(2);
        options.TokenValidationParameters.NameClaimType = "name";
        options.TokenValidationParameters.RoleClaimType = "roles";
    });
```

The second sketch is a different scheme, for a small API that issues its own tokens. The key bytes come from configuration. In Azure that configuration value is a Key Vault reference. The file in git has no key. HMAC and RS256 do not live on this same options object.

```
builder.Services.AddAuthentication()
    .AddJwtBearer("internal", options =>
    {
        options.MapInboundClaims = false;
        options.TokenValidationParameters = new TokenValidationParameters
        {
            ValidateIssuer = true,
            ValidIssuer = builder.Configuration["Tokens:Issuer"],
            ValidateAudience = true,
            ValidAudience = builder.Configuration["Tokens:Audience"],
            ValidateLifetime = true,
            ValidateIssuerSigningKey = true,
            ValidAlgorithms = new[] { SecurityAlgorithms.HmacSha256 },
            IssuerSigningKey = new SymmetricSecurityKey(
                Convert.FromBase64String(builder.Configuration["Tokens:SigningKey"]!)),
            ClockSkew = TimeSpan.FromMinutes(1),
            RoleClaimType = "roles"
        };
    });
```

RequireHttpsMetadata stays at its default, true, in production. A metadata document fetched over plain HTTP is a place an attacker would like to serve signing keys. When you replace TokenValidationParameters with a new object, you are responsible for every flag on that object. Mutating the existing object, as the first sketch does, keeps the handler's other defaults in place.

Questions and answers

Q1. What are the three parts of a JWT, and what is encrypted?

Answer. Header, payload, and signature, separated by dots. The header carries alg and usually kid. The payload carries the claims. The signature binds those two parts to a key. A normal access token is a JWS: signed, and the payload is only base64url-encoded. Anyone with the token can read the claims. A JWE is the encrypted form, five segments, and this API does not treat Entra access tokens as JWE. I keep secrets out of claims because the partner and the logs can decode a signed token.

Say this as a tech lead

I say 'signed, readable' in the first sentence. If the panel wanted encryption, they asked for JWE, and I tell them we did not put secrets in the payload.

Q2. What is the difference between an access token and a refresh token?

Answer. The access token is presented to the Orders API. It is short-lived. The API checks it locally. The refresh token is presented to the token endpoint by the BFF or by a confidential partner. Rotation issues a new refresh token and retires the old one. I store a hash of the current refresh token and revoke the family if a retired one reappears. I do not send the refresh token to the Orders API. I do not lengthen the access token so the client can skip refresh. A stolen access token should die at exp, and exp should be soon.

Say this as a tech lead

Sign-out that I control is 'refuse the next refresh'. A denylist of access tokens is the extra step when exp is too far away to wait.

Q3. Which claims do you authorize on for the Orders API?

Answer. After iss, aud, and exp succeed, I read scp for a delegated user and roles for an app-only partner. I read tenant_id for the business tenant, or I map azp to a tenant row when the token identifies a partner application. I store oid when I need a stable Entra user id. I treat sub as pairwise. I treat tid as the directory, and as the customer only when the product really is one directory per customer. The body cannot supply a stand-in for any of these.

Say this as a tech lead

I can point at the claim on a sample token from the test tenant. If I cannot, I have been authorizing a name I hoped was there.

Q4. Walk the validation steps in order.

Answer. Three segments. Allowed alg, and a refusal of none. A key I already trust, selected by kid among those keys. Signature. Issuer. Audience. Lifetime, including the clock skew I configured. Then the authorization policy reads scope, role, and tenant. Signature, issuer, audience, and lifetime failures are 401. A policy failure is 403. I never read tenant_id to choose a database and only afterwards check the signature.

Say this as a tech lead

The order is the whole answer. A candidate who starts from the role claim has skipped the part that makes the role believable.

Q5. What is the algorithm confusion attack?

Answer. The token header chooses an algorithm the API did not intend. alg none is the crude form. The sharper form is a token signed with HMAC using the API's public key as the secret, accepted by a validator that also allows RS256 and uses that public key for both. I pin RS256 on the Entra scheme and take keys only from metadata. I pin HMAC on a local issuer scheme and take the secret only from Key Vault. One TokenValidationParameters does not carry both. The token does not get to introduce a new key.

Say this as a tech lead

I mention alg none and the RSA-to-HMAC switch. Then I say the pin is in the options, so a header cannot widen it.

Q6. When would you choose a reference token instead of a JWT?

Answer. When I must revoke the credential in seconds and I can pay for introspection. The API receives an opaque handle and asks the issuer, or a cache of that answer. A JWT is the right default for the Orders API in front of Entra: local validation, no per-request call to the identity platform, revocation bounded by exp and by refresh rotation. If I cache introspection, I say the cache lifetime out loud, because a cache of an hour is a JWT with extra steps.

Say this as a tech lead

I pick JWT for this API. I describe the reference token as the revocation tool, with its round trip, and I do not pretend we already run one.

Q7. Walk the JwtBearer options you would ship.

Answer. Default scheme bearer. Authority from configuration, audience from configuration, MapInboundClaims false, HTTPS metadata on. Validate issuer, audience, lifetime, and signing key. ValidAlgorithms is RS256 for the Entra access tokens we have confirmed. Clock skew is an explicit value, two minutes in the sketch, or the five-minute default if I have not had a reason to change it. Role claim type is roles. A second scheme named internal is only for an API that issues its own tokens, with a symmetric key from Key Vault and HMAC pinned. The Orders partner API ships the first scheme.

Say this as a tech lead

I can write Authority, Audience, the four Validate flags, and the algorithm pin from memory. The rest I can look up. Those five are the ones a bad PR turns off.

Q8. A token validates and the action still returns 403. What do you check?

Answer. The signature, issuer, audience, and lifetime already passed, so this is the policy. I decode scp and roles with MapInboundClaims in mind. A space-separated scp fails a naive RequireClaim equality. A partner client-credentials token may have roles and an empty scp. tenant_id may be missing, and the policy requires it. I compare the policy name on the endpoint with the policy I think I tested. 403 with a correct scope usually means I authorized the wrong claim type, often the long URI form, or I required both a group policy and a route policy.

Say this as a tech lead

I ask for one real token and the policy name. I do not loosen validation to make the 403 go away.

Q9. Why is a long-lived access token the wrong fix?

Answer. The API will accept it until exp, on every instance, with no call back to Entra. Sign-out, a disabled user, and a removed role wait for that exp unless I add a denylist. The token shows up in places operators copy. Refresh rotation already solves the client experience: the access token stays short, the BFF refreshes it,

and a retired refresh token kills the session. I change the lifetime policy with the identity team when the short token is too short. I do not hard-code a multi-day exp in our own issuer to avoid that conversation.

Say this as a tech lead

The fix for a chatty client is a refresh on the BFF. The fix for a slow revoke is a shorter exp, or a denylist I am willing to operate.

Web API scalability

The Orders API is assumed to peak at about 50 requests a second. That number comes from the study guides. It is an assumption, and you say so. About 2 million orders a year is the other assumption. A year has 31,536,000 seconds, so 2,000,000 orders is about 0.063 orders a second on average, roughly one order every 16 seconds. Peak and average are different shapes. Most of the 50 requests are reads. If you assume four reads for every write, peak is about 40 reads and 10 writes a second. Label that split as an assumption too. The read target in these guides is p95 under 300 milliseconds. Email is within a minute, off the request.

If a contract says 99.9 percent over a 30-day month, the arithmetic is 30 times 24 times 60, which is 43,200 minutes, times 0.001, which is about 43 minutes. That is a budget you can say out loud. It is not the Orders API's measured availability. You spend those minutes on a failover, a bad deploy, or a dependency. A retry storm spends them for you.

Stateless instances

A stateless API keeps the durable facts outside the process. The order is in Azure SQL. The idempotency row is in Azure SQL. A shared cache, if you add one, is Redis or another store every instance can see. The process may keep a short-lived memory cache of a public catalog, and it may lose that cache on recycle. The next request still works.

Sticky sessions exist to hide state you stored by accident: an in-memory session, a static cart, a dictionary of locks. ARR affinity pins the caller to one App Service instance. The pin survives after that instance is sick, and it defeats the even spread you wanted when you scaled out. Move the state out, then turn affinity off. Affinity is not the design for POST /orders.

Horizontal scale adds instances of the same site. Each instance has its own memory, its own static fields, and its own connection pool. A lock in a static object covers one process. The second instance has its own lock and will enter the same critical section. Shared coordination is a unique index, a SQL row, or a Service Bus session you chose on purpose.

State	Where it lives at 2 or more instances
Order, lines, idempotency key	Azure SQL, in one transaction
User session for the Angular app	BFF cookie backed by a server store, or a distributed cache
Catalog cache	Per-process output cache, or Redis when every instance must agree
Partner rate-limit counters	In-process if the limit is per instance and you say so; a gateway when the budget is global
Email send	Service Bus, then the Function, with a processed-message row
Lock around 'create order'	Unique index on the idempotency key, not a Dictionary

Async, the thread pool, and sync-over-async

ASP.NET Core runs requests on the thread pool. An await on SQL or HTTP releases the thread for the duration of the IO. The thread returns when the operation completes. That is async all the way: the action is async, EF is ToListAsync or SaveChangesAsync, HttpClient is awaited, and the CancellationToken from the request is passed into each of them.

Sync-over-async is .Result, .Wait(), or GetAwaiter().GetResult() on work that is already asynchronous. The thread stays borrowed while the IO runs. Under a context that needs that same thread to finish the operation, it can deadlock. Under load, it starves the pool. The pool adds threads slowly. Requests queue inside the server. Latency climbs while CPU still looks calm. That chart is the clue.

A small illustration, not a measurement of this API. At 50 requests a second, a call that blocks a thread for 200 milliseconds of SQL holds about 10 threads on average, because 50 times 0.2 is 10. A call that blocks for a hung 10 second dependency represents 50 times 10, or 500 threads of waiting. The pool will not grow to 500 on the spot. New requests wait, and the probe can time out. The read target remains p95 under 300 milliseconds, with the thread released during the SQL wait.

```
[HttpPost("/orders")]
public async Task<IActionResult> Post(CreateOrderRequest body, CancellationToken ct)
{
    var tenantId = RequireTenant(User);
    var result = await handler.HandleAsync(body.ToCommand(tenantId), ct);
    return ToActionResult(result);
}

// The bug in review:
// var order = handler.HandleAsync(command, ct).Result;
```

Interview trap

Wrapping a synchronous block in Task.Run inside the request. That still occupies a pool thread, and now you have two. Also catching every exception from .Result and returning 500, which hides the deadlock until the next spike. The fix is async to the edge, including the filter and the middleware you added.

Timeouts, retries, jitter, and a circuit breaker

Every outbound call has a timeout shorter than the caller's patience and shorter than the gateway's timeout, so you return a controlled error. Retry only when the call is safe to repeat. A GET of a catalog price can retry. A POST that creates a charge retries only when that call carries an idempotency key the other side honors. Jitter spreads retries so a blip does not become a synchronized stampede. A circuit breaker opens after a failure ratio you chose, and it fails fast for a cool-down so the sick dependency can recover.

The numbers below are an example policy, the same kind of sketch the deep dive uses. They are not a quota. You set them from the dependency's latency and the user's budget, and you review them like code.

```
builder.Services.AddHttpClient<CatalogClient>(client =>
{
```

```
    client.BaseAddress = new Uri(builder.Configuration["Catalog:BaseUrl"]!);
    client.Timeout = Timeout.InfiniteTimeSpan;
})
.AddStandardResilienceHandler(options =>
{
    options.AttemptTimeout.Timeout = TimeSpan.FromSeconds(2);
    options.TotalRequestTimeout.Timeout = TimeSpan.FromSeconds(5);
    options.Retry.MaxRetryAttempts = 2;
    options.Retry.UseJitter = true;
    options.CircuitBreaker.FailureRatio = 0.5;
    options.CircuitBreaker.MinimumThroughput = 20;
});
```

HttpClient.Timeout is disabled here so it does not race the attempt timeout inside the resilience handler. The attempt timeout is the per-try budget. The total timeout bounds the tries together. MinimumThroughput of 1 would open the breaker on a single blip. SQL uses a command timeout and EnableRetryOnFailure for transient errors, including a brief Azure SQL failover. The retry must re-run a unit of work that is safe to re-run. The idempotency unique key is what makes a retried insert safe. When the policy is exhausted, return 503 or a problem document. Log the dependency name and the correlation id.

Caching

Response caching follows HTTP cache headers. A public catalog GET can send Cache-Control: public with a short max-age. An order GET sends Cache-Control: no-store. A shared cache that stores /orders/10 without varying on the caller will serve tenant A's order to tenant B. Authorization headers usually keep shared caches from storing the response. Set the header you mean, and do not rely on a default you have not read.

Output caching remembers the generated response on the server. The built-in store is in the process, so ten instances are ten caches. Use it for an anonymous catalog response with a short expiry. An order response is personalized. If you output-cache it, the vary key includes the tenant claim. A cache entry whose key omits the tenant is a data leak that scales with your hit rate.

IDistributedCache, usually Redis, is the store every instance shares. The key includes the tenant and the id. You choose the TTL. Reference data that is the same for many callers, and a tenant's own lookup with the tenant in the key, are the safe cases. An order total that a second POST can change needs a TTL you can explain, or you delete the key on write. A cached authorization decision needs the user or the client id in the key.

A cache stampede is many requests missing the same key at once and all running the SQL. It happens on expiry and on a cold start after a deploy. At the assumed 50 requests a second, a single popular catalog key that expires can turn into 50 identical queries in that instant. Mitigate with a single-flight lock per key, a slightly random TTL so keys do not expire together, or by serving the previous value while one request refreshes. Across instances the lock is a short-lived Redis set-if-not-exists, not a static Dictionary.

```
builder.Services.AddOutputCache();
builder.Services.AddStackExchangeRedisCache(options =>
{
    options.Configuration = builder.Configuration["Redis:Connection"];
});
```

```
app.MapGet("/catalog/{sku}", GetSku)
    .CacheOutput(p => p.Expire(TimeSpan.FromSeconds(30)));

var key = $"orders:{tenantId}:{orderId}";
var cached = await cache.GetAsync(key, ct);
```

The catalog route is anonymous reference data. GET /orders stays out of that policy. The Redis connection string is configuration, not a value in the git file. HybridCache, from .NET 9 onward, is the API for a local cache in front of a distributed cache. The key still includes the tenant.

Pagination, cancellation, bounded concurrency, idempotency

Keyset pagination seeks from the last (CreatedUtc, Id) the client saw. Offset asks SQL to read and discard the skipped rows. A deep offset at 50 requests a second is a scan you scheduled on purpose. Cap the page. The SQL section writes the keyset statement. The API contract is the cursor, not a page number the client invents by multiplying.

Pass the request CancellationToken to EF and to HttpClient. A caller who disconnects should stop the SQL command. Catching the cancellation and returning 500 turns a hang-up into an error budget incident. Link a shorter timeout only when you mean a budget inside the request:

```
using var budget = CancellationTokenSource.CreateLinkedTokenSource(ct);
budget.CancelAfter(TimeSpan.FromSeconds(2));
var page = await queries.ListAsync(tenantId, cursor, budget.Token);
```

Bounded concurrency is the cap on fan-out inside one request. Fifty lines, each calling a tax service with no limit, is 50 outbound calls per order. At 10 order writes a second that is 500 outbound calls a second, and that split was already an assumption. Parallel.ForEachAsync with a small MaxDegreeOfParallelism is the local cap. The resilience handler's timeout and breaker are the other cap. Queue limit zero on a rate limiter rejects with 429 instead of parking work on the thread pool.

Idempotency under concurrency is a unique key on (TenantId, KeyValue), inserted in the same transaction as the order. Two retries arrive together. One insert wins. The other gets the unique violation, reads the stored row, and returns it when the request hash matches. A different hash is 409. An in-memory dictionary of keys works on one instance during a demo and creates two orders on two instances.

```
await Parallel.ForEachAsync(lines, new ParallelOptions
{
    MaxDegreeOfParallelism = 4,
    CancellationToken = ct
}, async (line, token) =>
{
    line.Tax = await tax.QuoteAsync(line.Sku, line.Quantity, token);
});
```

N+1 at the API boundary

N+1 is one query for the page and one query per row. EF does it when you loop and touch a navigation that was not loaded, or when the Angular app calls GET /orders/{id}/lines for every row of a list it already displayed. The fix at the API is one shaped query: a projection, or Include with AsSplitQuery when a join would multiply rows, or a SQL statement you wrote. AsNoTracking belongs on the read. Tracking every line of a list allocates a graph you will not save.

```
var page = await db.Orders.AsNoTracking()
    .Where(o => o.TenantId == tenantId)
    .OrderByDescending(o => o.CreatedUtc).ThenByDescending(o => o.Id)
    .Take(50)
    .Select(o => new OrderListItem(o.Id, o.Number, o.Total, o.Status))
    .ToListAsync(ct);

var detail = await db.Orders.AsNoTracking()
    .Where(o => o.TenantId == tenantId && o.Id == id)
    .Include(o => o.Lines)
    .AsSplitQuery()
    .SingleOrDefaultAsync(ct);
```

Drop to SQL when the LINQ still produces a loop, a client-evaluated filter, or a plan you cannot explain. The list endpoint returns the columns the screen needs. The Angular client renders that payload. It does not issue one HTTP call per order id to fill the grid.

Backpressure

429 means the caller is over a budget you defined. Return Retry-After when you have a retry interval. 429 is not 500 and not 503. A limiter with a queue of thousands accepts work you cannot do and holds threads. Queue limit zero fails the extra call immediately. An in-process window is per instance. At two instances, a partner can get about twice the per-instance budget. Say that. A global budget lives at a gateway in front. Turning on ARR affinity to make the memory counter look global pins the partner to one instance.

Unbounded fan-out is the other failure. One request starts a task per line, per shipment, and per notification, with no cap, and each task opens SQL. The assumed 50 requests a second become thousands of concurrent commands. Bound the parallelism. Cap the page. Let Service Bus carry the work that can finish after the 201.

```
builder.Services.AddRateLimiter(options =>
{
    options.RejectionStatusCode = StatusCodes.Status429TooManyRequests;
    options.AddFixedWindowLimiter("partner", limiter =>
    {
        limiter.PermitLimit = 100; // team choice, per instance, not an Azure default
        limiter.Window = TimeSpan.FromMinutes(1);
        limiter.QueueLimit = 0;
    });
});
```

What does not scale

- A static Dictionary used as a lock or as the idempotency store. The second instance has an empty one.
- A static mutable list used as a catalog cache. It tears under concurrency and diverges across instances.
- SaveChanges, or an HTTP call, inside a loop over order lines when one set-based write would do.
- The Angular app calling SQL indirectly once per row: a list of ids, then a GET per id.
- A HostedService timer inside the API that sends mail. It runs once per instance.
- An unbounded SqlClient pool times the maximum instance count. That product is the connection count you compare with the Azure SQL limit document.

The SqlClient default maximum pool size is 100 per process. Confirm the connection string you ship, because a teammate may have raised it. If the plan maximum is 8 instances, 8 times 100 is 800 connections from this app, before the Function app opens its own pools. You compare 800 with the connection limit on the current Azure SQL tier. You look up the limit. You do not recite one from memory.

Questions and answers

Q1. What does stateless mean for the Orders API, and when do sticky sessions show up?

Answer. Stateless means any instance can serve the next request. The order, the idempotency key, and the outbox row are in Azure SQL. The process keeps no cart and no session. Sticky sessions show up when someone stored that state in memory and then turned on ARR affinity so the same instance would see it. I move the state out and turn affinity off. A load balancer pin is a temporary patch during an incident, and I write down the date I will remove it.

> **Say this as a tech lead**
>
> The question I ask of every field is: what happens on the second instance? If the answer is we only run one, the follow-up is what happens when the platform restarts that one.

Q2. What has to live outside the process when you scale App Service from one instance to several?

Answer. Session, a cache that must be coherent, distributed locks, and anything a timer does exactly once. Files on local disk are per instance. Static fields are per process. The SQL connection count becomes pool size times instance count, and that has to fit the database tier. Rate limits kept in memory become per-instance limits, which is acceptable only when the contract says so. The email worker stays on Service Bus so it does not scale just because the API scaled, and it does not starve the API when the mail provider slows down.

> **Say this as a tech lead**
>
> I size the pool against the maximum instance count on the plan, not against the two instances we happen to be running today.

Q3. Explain async all the way and thread-pool starvation.

Answer. The request thread should be busy only while we run CPU. Await releases it during SQL and HTTP. At the assumed 50 requests a second, blocking for a 200 millisecond call holds on the order of 10 threads. Blocking for a 10 second hang holds on the order of 500 threads of wait, and the pool grows slowly, so

requests queue and latency rises before CPU does. I pass CancellationToken into EF and HttpClient. I keep controllers, handlers, and middleware async. The 200 milliseconds and 10 seconds are illustrations. The read target I will defend is p95 under 300 milliseconds.

Say this as a tech lead

If p95 is high and CPU is low, I look for blocked threads and a slow dependency before I add instances. More instances of a blocked thread multiply the blocked threads.

Q4. What is sync-over-async, and why is Task.Run not the repair?

Answer. Sync-over-async blocks a thread on asynchronous work with Result, Wait, or GetResult. On ASP.NET Core it can deadlock when a single-threaded context is waiting for itself, and it starves the pool when it merely blocks. Task.Run moves the block onto another pool thread. The request still occupies a thread until the inner work finishes, and you have added scheduling. The repair is HandleAsync awaited from an async action, with the same CancellationToken.

Say this as a tech lead

In review, Result and Wait on an async call are a blocking comment. I ask for the await, not for a wrapper.

Q5. How do timeouts, retries, jitter, and a circuit breaker fit together?

Answer. The attempt timeout bounds one try. The total timeout bounds the whole pipeline. I retry idempotent calls, a small number of times, with jitter, so a blip does not wake every instance in the same millisecond. I do not retry POST /orders from a client that has no idempotency key. The breaker opens after a failure ratio and a minimum throughput I can say, and then it fails fast. The sketch in this section is an example policy: two retries, a two-second attempt, minimum throughput 20. SQL transient retries go through the execution strategy and a unique idempotency key. When the budget is gone I return 503.

Say this as a tech lead

Retries without a budget attack the dependency that is already sick. I would rather fail inside a few seconds than hold a thread for half a minute.

Q6. Response cache, output cache, or Redis. Which one, for what?

Answer. Response caching is HTTP. Public catalog responses can be cached by intermediaries for a short max-age. Order responses are no-store. Output caching stores the server's response. The default store is the process, so it is a latency help for anonymous GETs, and it is not one shared picture of a tenant's orders. Redis is the shared store. The key includes tenant id and the entity id. I cache reference data, and I cache a tenant's read model only with a TTL or an invalidation story. I leave authorization results out of a key that omits the caller.

Say this as a tech lead

If I cannot say who is allowed to receive a cached byte, I do not cache it. A cross-tenant hit is worse than a slow query.

Q7. What is a cache stampede, and what do you do about it?

Answer. Many requests observe a miss at the same moment and all run the backing query. Expiry of a hot catalog key, or a deploy that empties memory, does it. At about 50 requests a second that key can produce a burst of identical SQL. I single-flight the refresh per key, add jitter to the TTL, or serve a stale value while one caller refreshes. The cross-instance lock is a short Redis lock. A static lock object single-flights only the current process.

> **Say this as a tech lead**
>
> I mention stampede when I propose a one-minute cache on a popular key. The cache is a kindness to SQL only if the miss path stays single.

Q8. How do pagination, cancellation, and idempotency behave under load?

Answer. The list is a keyset on CreatedUtc and Id, with a cap, matching the index in the SQL section. Offset on a deep page reads the skipped rows on every request. CancellationToken flows to EF and HTTP so a disconnected client aborts the command. Two POSTs with the same idempotency key hit a unique index inside the order transaction. One wins, the other replays the stored response when the body hash matches. An in-memory set of keys does not survive the second instance or the recycle.

> **Say this as a tech lead**
>
> Paging, a maximum page size, and the idempotency unique key are acceptance criteria. I do not schedule them for a later hardening sprint.

Q9. Where does N+1 show up, and when do you drop to SQL?

Answer. It shows up as a list query plus a query per order for lines, often from a loop in C# or from the Angular client calling one endpoint per row. I project the list with AsNoTracking and a Take. I load one order's lines with Include and AsSplitQuery when the join would duplicate parent columns. I drop to a single SQL statement when I still cannot get one round trip or the plan is wrong. The client receives a page DTO. It does not reassemble the page with fifty GETs.

> **Say this as a tech lead**
>
> I look at the SQL trace for a page load, not only at the C#. Fifty similar commands for one screen is the finding.

Q10. How do you apply backpressure, and what is unbounded fan-out?

Answer. Backpressure is a 429 with Retry-After, from a limiter whose queue length is zero, so extra calls fail instead of waiting. A fixed window of 100 per minute per instance is a team choice I label as a choice, the same way the other notes in this repo do. It is not an Azure default. Two instances mean two windows. A shared budget is a gateway. Unbounded fan-out is one request that starts unlimited parallel SQL or HTTP. I cap it with Parallel.ForEachAsync, a small degree of parallelism, and the same cancellation token. Work that can finish later goes to Service Bus.

Say this as a tech lead

I would rather return 429 to one partner than let that partner hold the SQL connection pool for everyone else.

Q11. Name the designs that fall over when the second instance arrives.

Answer. An in-memory dictionary of idempotency keys or locks. A static mutable cache. A timer hosted in the API. ARR affinity left on to make those bugs look stable. Chatty EF that saves once per line. An Angular screen that calls the API once per cell. A connection pool copied unchanged onto a higher instance maximum. Each of those is correct on a single process and wrong at the assumed peak once you scale out for availability.

Say this as a tech lead

Two instances is my minimum when I want a deploy or a fault to leave one process up. That is an availability choice. The code has to deserve it.

Q12. At about 50 requests a second, what do you actually size?

Answer. I label 50 a second as the study assumption, mostly reads, with about 2 million orders a year underneath it, which is about one order every 16 seconds on average. I watch p95 against the 300 millisecond read target, CPU, the thread-pool queue, dependency duration, and SQL. I compare pool size times maximum instances with the Azure SQL connection limit in the tier document. I do not invent an instance count. I add an instance when the measurement says the process is the bottleneck, and I keep two when I need one to survive a recycle. If someone cites 99.9 percent, I translate a 30-day month into about 43 minutes and ask which failures are allowed to spend that budget.

Say this as a tech lead

I size from the chart and the limit document. I can do the 43-minute arithmetic in the room. I cannot do a fake capacity number.

Azure, in more depth

The Orders API runs on App Service. The mail worker is an Azure Function on the isolated worker. Azure SQL stores the rows. Service Bus carries the committed fact. Key Vault holds secrets. Entra ID issues the caller's token. The process uses a managed identity. This section is the follow-up on those pieces. Prices, SKU quotas, and cold-start milliseconds stay in the current document for the service. You name the document. You do not recite a number you have not opened.

App Service

The plan is the SKU you are paying for: the features it includes, the size of an instance, and how many instances you may run. Scale up changes the size. Scale out adds instances of the same site. Slots, Always On, and a useful instance maximum exist on tiers that include them. You confirm that on the plan in front of you.

Always On keeps the process loaded when traffic is quiet, which is what you want for an API with a p95 target. ARR affinity pins a client to an instance with a cookie. Turn it off for this stateless API. A health check path is the URL App Service probes. Several failures take the instance out of rotation, and continued failure can replace it. Point that path at a cheap live check that the process can answer. A ready check that opens SQL is for operators and for a gateway that can drain an instance without deleting it. If you point the App Service probe at SQL, a database blip can remove the fleet. Say that choice if you make it.

A staging slot runs the new bits. You warm it, you hit the live check, and you run a smoke call. Swap exchanges the slots. The previous bits remain on the slot you swapped out of, and swap-back is real when those bits still understand the current schema. A setting marked as a slot setting stays on its slot. A setting without that mark moves with the bits. Connection strings that point at a different database are slot settings on purpose, or both slots are given the production-like string before the swap. The deep dive's release order still holds: expand the schema, deploy to the slot, smoke, swap, swap back only while the old bits can read the new schema.

```
Release for a new nullable column the new code reads:
1. Pipeline applies the expand script with the migrator identity.
2. Deploy bits to the staging slot. Slot settings do not point this slot
   at a database the smoke test must not touch.
3. Warm the slot. GET /health/live returns 200 without depending on SQL.
4. Smoke a read on the slot hostname.
5. Swap. Watch error rate, p95, and SQL dependency time.
6. Swap back if the chart moves and the previous bits still match the schema.

A dropped column in the same release makes step 6 a lie.
```

System-assigned identity is born with the slot and is a different principal from production. User-assigned identity is a resource you attach. The SQL user and the Key Vault access policy have to exist for the identity that will serve traffic after the swap. An identity that was granted only on the staging slot fails the first query once production is serving those bits.

Azure SQL

The app connects with Authentication=Active Directory Managed Identity, Encrypt required. An admin creates a user from that identity and adds it to the runtime role. There is no SQL password in the string. EnableRetryOnFailure covers transient errors the provider treats as retryable, including a brief failover. The retry re-runs the whole operation. A transaction you begin yourself goes through CreateExecutionStrategy, so the retry starts the transaction over. The idempotency unique key makes that retry safe. A retry of a bare insert, with no unique key, can duplicate the order.

DTU is a bundled performance model. vCore is a more explicit choice of compute, with storage and IO you select alongside it. In the room you say which signal you have: CPU, IO, connection count, or the bill. Then you open the limit document for that tier. You do not pick a SKU from memory, and you do not quote a price.

The firewall is a public-network allow list. App Service outbound addresses can change as you scale. Allowing every Azure service is a wide door. A private endpoint is the conceptual alternative: the public path can be closed, the name resolves to a private address, and the app reaches SQL through the virtual network. You describe that shape. You look up the current private-link steps when you build it. The user's Entra token is still not the SQL login. The managed identity is.

```
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("Orders"), sql =>
    {
        sql.EnableRetryOnFailure(maxRetryCount: 5);
    }));

var strategy = db.Database.CreateExecutionStrategy();
await strategy.ExecuteAsync(async () =>
{
    await using var tx = await db.Database.BeginTransactionAsync(ct);
    db.Orders.Add(order);
    db.IdempotencyKeys.Add(keyRow);
    await db.SaveChangesAsync(ct);
    await tx.CommitAsync(ct);
});

-- Once, as an admin, not at app startup:
-- CREATE USER [id-api-prod] FROM EXTERNAL PROVIDER;
-- ALTER ROLE app_orders_runtime ADD MEMBER [id-api-prod];
```

Service Bus

Peek-lock delivers a message under a lease. Complete removes it after the work commits. Abandon makes it visible again. After the delivery count you configured, Service Bus dead-letters it. You alert when the dead-letter count is above zero, you read the reason, and you have a replay. The handler is idempotent because a lock that expires produces a second delivery. Keep the work inside the lock, or renew the lock. The lock duration is a queue property you read on the queue. You do not memorize a default in the interview.

Duplicate detection drops a second message with the same MessageId inside a window configured on the queue. Outside that window the same id is a new message. A partner who retries tomorrow is outside a

short window. The business guarantee remains the unique idempotency key in SQL. Duplicate detection is a short-range convenience for a publisher that retries immediately.

Sessions process messages that share a session id one at a time, in order, with a single active receiver for that session. Use a session when the events of one order must not run in parallel. The email worker usually skips sessions: each message is independent, and the processed-message row covers the duplicate. Sessions cost parallelism. Say the cost when you turn them on.

Prefetch asks the client to pull extra messages and hold their locks before the handler is ready. A high prefetch and a slow mail provider pin messages on one instance. Other instances sit idle. Locks expire. The messages reappear and you have manufactured duplicates. Start with a small prefetch, or none, and raise it when the handler is faster than the network and you have measured the lock.

Functions, secrets, and telemetry

New functions use the isolated worker. Your code runs in a separate process from the Functions host. The HTTP API stays on App Service because people are waiting and the read has a p95 target. The Function owns the mail because that work scales with the queue and must be allowed to retry without occupying API threads. Consumption can scale to zero, and the next invocation pays a cold start. That plan is a candidate when the mail still arrives inside the minute you assumed, and you have measured the cold start. A plan that keeps an instance warm is the candidate when a person is waiting. You decide on that question. You do not decide on a price you guessed.

A Key Vault reference in an App Service setting resolves to the secret at runtime, using the managed identity. The secret value is not in appsettings.json. That file may hold non-secrets: the audience, queue names, and timeouts you have chosen. It does not hold client secrets, SQL passwords, storage keys, signing keys, or connection strings that embed those. Local development uses user secrets or a developer vault. Production instrumentation uses a connection string in an app setting too.

```
// App setting, value stored in Azure, not in git:
// ConnectionStrings__Orders =
//   Server=tcp:orders.database.windows.net,1433;Database=orders;
//   Authentication=Active Directory Managed Identity;Encrypt=True

// Pricing__Rate =
//   @Microsoft.KeyVault(SecretUri=https://orders-kv.vault.azure.net/secrets/pricing-rate/)
```

Trace the request, the HTTP dependency, and the SQL dependency, and keep the correlation id on the log line. A trace attribute for an order id is useful on that one request. A metric dimension is a series. Order id as a dimension creates a series per order and will blow the metric bill and the query. Dimension on a low-cardinality value such as the route, the status code, or the dependency name. Tenant id joins order id in the danger list once you have many tenants. Put it on the trace or the log.

```
builder.Services.AddOpenTelemetry()
    .ConfigureResource(r => r.AddService("orders-api"))
    .WithTracing(tracing => tracing
        .AddAspNetCoreInstrumentation()
        .AddHttpClientInstrumentation()
```

```
        .AddSqlClientInstrumentation())
    .WithMetrics(metrics => metrics
        .AddAspNetCoreInstrumentation()
        .AddRuntimeInstrumentation());

Activity.Current?.SetTag("order.id", orderId.ToString());
ordersPlaced.Add(1,
    new KeyValuePair<string, object?>("result", "created"));
```

Front Door, API Management, and a BFF

You do not add all three because the diagram looks complete. A BFF is the only host the Angular app calls. It holds the cookie session and the tokens, and it calls the Orders API. API Management belongs when partners need a published product: a front door for keys, quotas, and a stable external contract. Front Door belongs when you need a global entry, TLS at the edge, or a WAF in front of public traffic. The Orders partner API can sit behind API Management later. The first-party app can sit behind a BFF later. The API still validates the token itself. A gateway that checked the token is a help. It is not the only check.

Piece	Add it when	Leave it out when
BFF	A browser must call something that can hold tokens	The only caller is a confidential partner
API Management	External partners need a product, a quota, and a key surface	One internal caller already has a contract with the API
Front Door	You need a global edge or a WAF in front of the public site	A single-region App Service is the whole public surface and you have no edge requirement yet

Interview trap

A slot swap after a column drop, with swap-back offered as the safety net. A sticky connection string that still points at staging after the swap, or a staging string that was not sticky and moved into production. An App Service health probe that opens SQL and then replaces instances during a failover. Duplicate detection described as the order idempotency guarantee. A metric tagged with order id. A price or a connection quota recited from memory.

Questions and answers

Q1. What is an App Service plan, and what does scale out change?

Answer. The plan is the tier: instance size, how many instances you can run, and whether slots and Always On are included. I confirm those on the plan we pay for. Scale up replaces the size. Scale out adds instances. Each instance has its own memory and its own pool. At the assumed peak of about 50 requests a second I do not invent an instance count. I watch CPU, the request queue, and SQL, and I keep at least two instances when I want one recycle to leave the site up. That two is an availability choice.

Say this as a tech lead

I separate the plan limit from the traffic assumption. The assumption is 50 requests a second. The plan maximum is whatever the portal shows, and the code has to be correct at that maximum.

Q2. Walk a deployment slot, a swap, and a sticky setting.

Answer. Staging runs the new bits. I warm it and smoke it on the slot hostname. Swap moves the bits into production and leaves the previous bits on the other slot. A slot setting stays behind. A normal app setting swaps. I use a slot setting for a value that must not follow the bits, such as a staging-only secret. I do not accidentally leave the production database string as a sticky staging value, and I do not let a staging database string swap into production. Schema changes are a pipeline step before traffic moves.

Say this as a tech lead

I can say which settings on our app are slot settings and why. If I cannot, I am not ready to swap.

Q3. Why is ARR affinity off, and why is Always On on?

Answer. The API is stateless, so any instance may take the next call. Affinity would pin a partner to a dead instance and would make an in-memory cache look correct. Always On keeps the worker process loaded when the site is idle, which protects the first request of the day and the p95 target. I turn affinity on only as a temporary patch while I move state out of the process, and I name the day it comes off.

Say this as a tech lead

Affinity off, Always On on, HTTPS only. Those three are the App Service sentence I can say without the portal in front of me.

Q4. What should the App Service health check path hit?

Answer. A live path that proves the process can answer, without opening SQL. App Service removes a failing instance and can replace it if failures continue. A probe that depends on SQL turns a database failover into a fleet replacement. I still expose a ready path that touches SQL lightly, for humans and for a gateway that can stop routing. Both paths are anonymous and return nothing about orders. The assumed read target is a reason to notice a bad instance. It is not a reason to tie process replacement to the database.

Say this as a tech lead

Live and ready are two URLs. I say which one the platform probes, and I say what a SQL outage does to that probe.

Q5. How does managed identity reach Key Vault and Azure SQL?

Answer. The app has a system-assigned or user-assigned identity. Key Vault grants that identity get on the secrets it needs. App settings hold references, and the platform resolves them. Azure SQL has a user created from the same identity, in app_orders_runtime, with DML and EXECUTE, not DDL. The connection string uses Active Directory Managed Identity. The identity that serves traffic after a swap is the one that was granted. A pipeline identity applies migrations. The caller's token is a third identity and is not the SQL login.

Say this as a tech lead

I draw the user, the app, and the pipeline every time. A design that uses one login for all three is the finding.

Q6. How do you retry Azure SQL without creating a second order?

Answer. EnableRetryOnFailure handles transient errors, including a short failover. I route my own transaction through the execution strategy so a retry repeats the transaction from the start. Inside that transaction I insert the order and the idempotency row. The unique key on tenant plus key means the second attempt conflicts, reads the winner, and returns the stored response. I do not retry by looping SaveChanges on a fresh order with a new id. Command timeout still applies. A deadlock victim, error 1205, retries only when that same story is safe.

Say this as a tech lead

The retry policy and the unique key are one design. A retry without the key is how a failover duplicates a payment.

Q7. DTU or vCore. What do you say in the room?

Answer. DTU bundles compute into one measure. vCore lets you talk about cores, memory, and storage more directly. I choose by opening the bill and the limit document: connections, CPU, IO, and storage for the workload we measured. I bring the connection product, pool size times maximum instances, as the number I compare with the tier's connection limit. I do not quote a price or a SKU quota from memory. If we are on the wrong model, the evidence is a limit we are hitting or a bill that does not match the shape of the work.

Say this as a tech lead

I say 'I would check the tier document' and I mean it. A memorized DTU number from a blog is how you lose the follow-up.

Q8. How do you think about the SQL firewall and private access?

Answer. The firewall is who may open the public endpoint. App Service outbound IPs are a brittle allow list because they can change when the stamp changes. A rule that allows all Azure services is broad. Private access means a private endpoint, the public path closed, and the app reaching SQL through the virtual network, with name resolution that follows. I describe that. I look up the current steps when I implement them. The private path does not replace the SQL permissions. The managed identity is still a user in a runtime role.

Say this as a tech lead

Network reach and SQL permission are two gates. I want both. Opening the firewall to the world and hiding the password is one gate, and it is the wrong one.

Q9. Walk a Service Bus message from delivery to dead-letter.

Answer. Peek-lock gives the Function a lease. The handler writes its idempotent SQL, then completes. An exception abandons the message, and it becomes visible again until MaxDeliveryCount. Then it is dead-

lettered with a reason. I alert on dead-letter depth. I can replay after a fix. If the work can outlive the lock, I renew the lock or I split the work. I complete after the commit. Completing before the commit loses the mail on a crash. The API has already returned 201. The user is not waiting on this thread.

Say this as a tech lead

Complete after the side effect, abandon on failure, dead-letter after the count, alert, replay. That is the poison path I want on the runbook.

Q10. Why is duplicate detection not your idempotency story?

Answer. Duplicate detection is a window on the queue. A repeated MessageId inside the window is dropped. The same MessageId after the window is delivered. I read the window on our queue when I operate it. I do not memorize a default. A business retry the next day, a second publisher, and a message that was dead-lettered and resubmitted can all sit outside that window. The order's idempotency is the unique key in SQL, stored with the request hash, in the same transaction as the order.

Say this as a tech lead

I let Service Bus help with an immediate publisher retry. I let SQL decide whether this order already exists.

Q11. When do you enable Service Bus sessions?

Answer. When messages that share a session id must run one at a time and in order. An order's events are the usual session id. One active receiver owns that session, so parallelism drops to the number of active sessions, not the number of messages. The confirmation email in this system does not need a session if each message carries a message id we dedupe in SQL. I add sessions when a real ordering bug appears, and I say what parallelism I gave up.

Say this as a tech lead

Sessions are an ordering tool with a throughput cost. They are not a default on every queue because the portal offered a checkbox.

Q12. What goes wrong with a large prefetch?

Answer. Prefetch pulls messages onto one consumer and starts their locks before the handler runs. A slow mail provider plus a large prefetch holds a pile of messages on one instance. The other instances have nothing. Locks expire, Service Bus redelivers, and the idempotent handler spends its time proving it already sent the mail. I keep prefetch small until I have measured a handler that finishes well inside the lock.

Say this as a tech lead

I treat prefetch as a performance knob with a duplicate side effect. I do not copy a sample that sets it to hundreds.

Q13. When is the isolated worker the right host, and how do you choose Consumption?

Answer. Isolated worker is the model I target for new functions: our process, separate from the host, on the runtime that moves forward with .NET 10. It is the right host for the mail, a schedule, and short work behind

Service Bus. It is the wrong host for the whole CRUD API when callers expect the read to stay hot all day. Consumption can scale to zero, so the next call pays a cold start. I consider it when the work can still finish inside the assumed minute for email, and I measure that cold start. If a person is waiting on the call, I keep an instance warm on a plan that allows it. I do not quote the plan price.

Say this as a tech lead

The split is failure and latency. The API stays up for the user. The Function retries the mail. Cold start is acceptable only for the side that nobody is staring at.

Q14. What must never be committed in appsettings?

Answer. Client secrets, SQL passwords, storage account keys, signing keys, and any connection string that contains those. The Entra authority, the audience, queue names, and the timeouts we chose can live in the file because they are not secret. Key Vault references and the managed-identity SQL string belong in App Service settings. Local secrets stay in user secrets or a developer vault. A production Application Insights connection string is an app setting too. I also keep tokens and connection strings out of logs.

Say this as a tech lead

If a secret reached git, the fix is rotation, not a commit that deletes the line. The history still has it.

Q15. What do you trace, and why is order id a bad metric dimension?

Answer. I trace the ASP.NET Core request, outbound HTTP, and SQL, with one correlation id. I may tag the trace with the order id, because that tag lives on one trace. A metric dimension creates a time series per value. Order id means a series per order. Tenant id becomes the same problem as the tenant count grows. I dimension metrics on result, route, and dependency name. I put the order id on the log and the trace when I am debugging one order.

Say this as a tech lead

Cardinality is a bill and a query that stops returning. I say the rule in review when someone adds a tag 'just in case'.

Q16. When do you add Front Door or API Management, and how is that different from a BFF?

Answer. The BFF is the browser's backend. It owns the cookie and the tokens and calls the Orders API. API Management is for publishing the partner surface: products, quotas, and a place for partner credentials, when we have external consumers who need that. Front Door is the global edge and a WAF when the public entry needs one. The Orders API does not require all three on day one. Whichever one I add, the API still validates the bearer token. A gateway check is not a substitute.

Say this as a tech lead

I add a box when it owns a job I can name. A box that only forwards bytes is a thing I will debug at night for no gain.

Q17. Tell the rollback story when a slot swap goes wrong.

Answer. I look at the deploy marker, the error rate, and the dependency time. If the previous bits still match the schema, I swap back first and diagnose second. Expand-only migrations are what make that true: a new nullable column, a new table, no rename and no drop in the same release. If this release dropped a column the old bits read, swap-back fails closed and I fix forward with a plan I say out loud. I send a one-line status while I do it. I do not scale out to hide a bad query that arrived with the bits, and I do not spend the first half hour in the editor when the slot can still reverse the change.

> **Say this as a tech lead**
>
> Reversible means the previous process can run on the current database. I decide that before the swap, not during the incident.

Q18. Production p95 jumped after the swap. What do you open, in order?

Answer. The correlation id from one slow request, then the chart: rate, latency, errors, SQL time, HTTP dependency time, CPU. The study target I compare against is p95 under 300 milliseconds for the read, labeled as the guide's target. If SQL time moved and the old bits are safe, I swap back. If a dependency is timing out, I look at retries and the breaker before I add instances. If CPU is high and SQL is flat, I look for a hot loop or thread-pool starvation. Scaling out during a lock storm adds waiters. I write the customer impact and the guard after the service is calm.

> **Say this as a tech lead**
>
> Metrics, a reversible action, then the plan or the code. The panel is listening for that order.

SQL questions and sample queries

The panel is not asking you to be the DBA. They are asking you to read a plan, pick an index with a write cost, and write a predicate that stays inside one tenant. The schema below completes the Orders example so the statements in this section have somewhere to land. Confirm the plan on your own SQL Server or Azure SQL before you argue about row counts. This guide explains the shape. It does not ship a result grid.

Schema

Orders is clustered on a bigint identity. That key is narrow and ever-increasing, so inserts land at the end of the leaf. TenantId is on every tenant-owned row. The hot list index matches the API keyset: tenant, then CreatedUtc, then Id, with the columns the list paints included at the leaf. The idempotency primary key is the business guarantee the scale section relies on.

```
CREATE TABLE orders.Tenants
(
    TenantId int           NOT NULL,
    Name     nvarchar(200) NOT NULL,
    CONSTRAINT PK_Tenants PRIMARY KEY CLUSTERED (TenantId)
);

CREATE TABLE orders.Customers
(
    CustomerId  int IDENTITY(1,1) NOT NULL,
    TenantId    int               NOT NULL,
    Email       nvarchar(320)     NOT NULL,
    DisplayName nvarchar(200)     NOT NULL,
    DeletedUtc  datetime2(3)      NULL,
    CONSTRAINT PK_Customers PRIMARY KEY CLUSTERED (CustomerId),
    CONSTRAINT FK_Customers_Tenants FOREIGN KEY (TenantId) REFERENCES orders.Tenants (TenantId)
);

CREATE UNIQUE INDEX UX_Customers_Tenant_Email_Active
    ON orders.Customers (TenantId, Email)
    WHERE DeletedUtc IS NULL;

CREATE TABLE orders.Orders
(
    Id         bigint IDENTITY(1,1) NOT NULL,
    TenantId   int                  NOT NULL,
    CustomerId int                  NOT NULL,
    Number     nvarchar(40)         NOT NULL,
    Status     nvarchar(32)         NOT NULL,
    CreatedUtc datetime2(3)         NOT NULL,
    Total      decimal(18, 2)       NOT NULL,
    RowVersion rowversion           NOT NULL,
    CONSTRAINT PK_Orders PRIMARY KEY CLUSTERED (Id),
    CONSTRAINT FK_Orders_Tenants FOREIGN KEY (TenantId) REFERENCES orders.Tenants (TenantId),
    CONSTRAINT FK_Orders_Customers FOREIGN KEY (CustomerId) REFERENCES orders.Customers
(CustomerId)
```

```
);

CREATE NONCLUSTERED INDEX IX_Orders_Tenant_Created
    ON orders.Orders (TenantId, CreatedUtc DESC, Id DESC)
    INCLUDE (Status, Total, CustomerId, Number);

CREATE UNIQUE INDEX UX_Orders_Tenant_Number
    ON orders.Orders (TenantId, Number);

CREATE TABLE orders.OrderLines
(
    OrderLineId bigint IDENTITY(1,1) NOT NULL,
    OrderId     bigint               NOT NULL,
    TenantId    int                  NOT NULL,
    Sku         nvarchar(40)         NOT NULL,
    Quantity    int                  NOT NULL,
    UnitPrice   decimal(18, 2)       NOT NULL,
    CONSTRAINT PK_OrderLines PRIMARY KEY CLUSTERED (OrderLineId),
    CONSTRAINT FK_OrderLines_Orders FOREIGN KEY (OrderId) REFERENCES orders.Orders (Id)
);

CREATE NONCLUSTERED INDEX IX_OrderLines_Order
    ON orders.OrderLines (TenantId, OrderId)
    INCLUDE (Sku, Quantity, UnitPrice);

CREATE TABLE orders.IdempotencyKeys
(
    TenantId     int              NOT NULL,
    KeyValue     uniqueidentifier NOT NULL,
    RequestHash  binary(32)       NOT NULL,
    OrderId      bigint           NULL,
    StatusCode   int              NOT NULL,
    CreatedUtc   datetime2(3)     NOT NULL,
    CONSTRAINT PK_IdempotencyKeys PRIMARY KEY CLUSTERED (TenantId, KeyValue)
);

CREATE TABLE orders.IdempotencyRejections
(
    RejectionId bigint IDENTITY(1,1) NOT NULL,
    TenantId    int              NOT NULL,
    KeyValue    uniqueidentifier NOT NULL,
    RequestHash binary(32)       NOT NULL,
    RejectedUtc datetime2(3)     NOT NULL,
    CONSTRAINT PK_IdempotencyRejections PRIMARY KEY CLUSTERED (RejectionId)
);

CREATE INDEX IX_IdempotencyRejections_Key
    ON orders.IdempotencyRejections (TenantId, KeyValue, RejectedUtc);
```

The list index is the one the deep dive already uses, with CustomerId and Number added to the include list so the queries in this companion can be covered. Every included column is a write cost. Cover the list and the date-range report. Do not INCLUDE every future column because it might be selected.

Words you need before the questions

A seek jumps into the index at the first key that qualifies and stops at the end of the range. A scan reads all or most of a leaf. A scan of a tiny table is cheap. A seek that then does a key lookup per row, for a large range, can cost more than a scan. A covering index holds every column the query needs in the nonclustered leaf: the key columns, the included columns, and the clustering key, which rides along for free. The query then skips the lookup.

SARGable means the column stands alone on one side of the predicate, so the optimizer can seek. Equality, a range, and a small IN list are the usual shapes. A function on the column, a conversion of the column, or a leading-wildcard LIKE hides the key. Put the function on the constant. A half-open range, greater than or equal to the start and less than the end, keeps the column bare.

Under locking read committed, readers take shared locks and writers take exclusive locks, so they block each other. Read committed snapshot serves the statement a committed version of the row and readers do not take those shared locks. Writers still block writers. Snapshot isolation is a transaction-wide view as of the start, and a writer can fail with an update conflict. Both use a version store. Confirm the database setting before you recommend a change. A lost update, two sessions reading a total and both writing it back, still needs an atomic update or the RowVersion token. Versioning does not invent that for you.

Parameter sniffing is a plan cached for the first parameter values. A plan compiled for a tenant with a dozen orders is reused for a tenant with a huge share of the table, or the reverse. Estimated versus actual rows is how you see it. Query Store is the place to look on Azure SQL. A hint is a pin with an owner and a removal date.

Interview trap

SELECT * pulls columns that are not in the covering leaf, so the seek grows key lookups, and the optimizer may abandon the seek for a clustered scan. A function on CreatedUtc, or an OR across unrelated columns, does the same to a seek you thought you had. NOLOCK is not the standard fix for blocking. It can return torn or missing rows. Reach for the snapshot setting you have confirmed, a shorter transaction, and an index that locks only the rows you mean.

Questions and answers

Q1. Which index serves the tenant's newest orders, and why that column order?

Answer. IX_Orders_Tenant_Created on (TenantId, CreatedUtc DESC, Id DESC), including Status, Total, CustomerId, and Number. TenantId is the equality. CreatedUtc is the range and the order. Id is the tie-breaker so two orders in the same instant do not collapse. The clustered key is Id alone, which keeps inserts orderly and keeps every secondary index narrow. A nonclustered index that leads with CreatedUtc would seek a time range across every tenant and then filter, which is the wrong leading column for a multi-tenant list.

Say this as a tech lead

Equality columns first, then the range and the ORDER BY, then INCLUDE for the select list. I say the write I slow down when I add the include.

Q2. What is a covering index, and what does INCLUDE cost?

Answer. The nonclustered leaf contains every column the query outputs and filters, so SQL seeks that index and stops. Without Status and Total in the leaf, a list of those columns seeks and then key-looks up into the clustered index once per row. INCLUDE puts Status, Total, CustomerId, and Number on the leaf without making them part of the sort key. Each included column widens the leaf and slows the insert and the update of that column. I cover the hot list. I do not cover a report that runs once a night if it doubles the write path the partner feels.

> **Say this as a tech lead**
>
> I ask for the actual plan on production-like data before I add the index. An include list copied from SELECT * is how the index becomes a second copy of the table.

Q3. What does SARGable mean? Rewrite a filter on the year of CreatedUtc.

Answer. The column has to stay bare so a seek can start at a key. WHERE YEAR(CreatedUtc) = 2026 computes the year for every row and scans. The range form is CreatedUtc >= '2026-01-01' AND CreatedUtc < '2027-01-01'. The same bug in LINQ is o.CreatedUtc.Year == 2026. I write a range in the LINQ too. A trailing-wildcard LIKE can seek. A leading wildcard cannot. CONVERT(date, CreatedUtc) = @day is the same trap. I put the bounds on the parameter side: CreatedUtc >= @day AND CreatedUtc < DATEADD(day, 1, @day).

> **Say this as a tech lead**
>
> If the index exists and the plan still scans, I read the predicate before I scale the App Service. More instances run the scan more often.

Q4. Show an implicit conversion that turns a seek into a scan.

Answer. Number is nvarchar. A .NET string parameter is nvarchar, so WHERE TenantId = @TenantId AND Number = @Number can seek UX_Orders_Tenant_Number. WHERE Number = 1001 compares an nvarchar column to an int. Int has higher type precedence, so SQL converts the column, the seek is gone, and a non-numeric Number throws. The other classic is a varchar column compared with an nvarchar parameter: nvarchar wins, the column converts, and the index on that varchar column is not seekable. I match the parameter type to the column. I do not 'fix' it by wrapping the column in CONVERT.

> **Say this as a tech lead**
>
> I look at the plan's predicate for CONVERT_IMPLICIT on the column. That glyph is the scan. The C# type of the parameter is the fix.

Q5. A procedure is fast for one tenant and slow for another. What do you do?

Answer. I capture the statement, the parameters, and the duration from Query Store. I compare estimated rows with actual rows. A plan compiled for a small tenant and reused for a large one is parameter sniffing. I also check the opposite, and I check blocking, because a lock wait looks like a slow procedure when you run it alone and it is fine. I fix a non-SARGable predicate or a missing index first. OPTIMIZE FOR, a plan guide, or a targeted recompile is a pin I can name and a person who will remove. A local variable copied from the parameter hides the sniffed value and also hides a good estimate for the common tenant. OPTION

(RECOMPILE) on a query at 50 requests a second spends CPU compiling. That 50 is the study assumption, not a hint license.

Say this as a tech lead

The API symptom is one route, one tenant, bad p95. The SQL symptom is a plan. I do not scale out to hide it.

Q6. Compare read committed, read committed snapshot, and snapshot isolation.

Answer. Locking read committed: shared locks for readers, exclusive locks for writers, and they block. Read committed snapshot keeps statement-level read committed behavior and serves versions, so readers do not block writers. Writers still block each other on the same row. Snapshot isolation gives each transaction a view from its start, and a second writer can fail with a conflict it must handle. I read is_read_committed_snapshot_on before I recommend a change. Versioning uses a version store. On current Azure SQL I confirm whether accelerated database recovery holds it, and I do not recite a tempdb myth. RCSI does not make read-modify-write safe. I still update with a single statement, Total = Total + @delta, or I compare RowVersion and return 409.

Say this as a tech lead

I name the setting we run, not the setting I wish we ran. Turning RCSI on in the middle of an outage is a database change, and it gets a test plan.

Q7. What is a deadlock, and what do you change after error 1205?

Answer. Two sessions hold locks the other needs, in a cycle. SQL kills one with 1205. The EF execution strategy may retry when the operation is idempotent. The lasting fix is one lock order in every transaction, a short transaction, and an index that stops a scan from locking a wide range. The sample later in this section is Orders then OrderLines in both sessions. I read the deadlock graph: processes, resources, statements. Triggers and an ORM that updates every column create orders of locks nobody wrote down. A retry that succeeds with no graph leaves the cycle in place. I put a metric on 1205. A rising retry count is a ticket even when the caller eventually sees 200.

Say this as a tech lead

I retry only the safe unit of work. Then I remove the cycle. I do not hold the transaction open across the payment HTTP call, which is how the window gets long enough to hit.

Q8. When is a scan acceptable, and when is a seek a trap?

Answer. A scan of a small table, or of a range that really is most of the tenant, can be the honest plan. A seek is a trap when it performs a key lookup for every row of a large range, or when it seeks once per row from a nested loop the client caused. I look at the operator and the actual rows. I also look at the residual predicate. A seek on Id with a residual TenantId is the right plan for a point read, and it is still wrong as a security story if I forget the tenant predicate and seek only on Id.

Say this as a tech lead

I refuse 'we need a seek' as a slogan. I want the operator, the rows, and the columns. Then I decide.

Q9. What do you compare on an estimated plan versus an actual plan?

Answer. Estimated rows against actual rows, on the same operator. A gap of orders of magnitude means statistics, a sniffed parameter, or a predicate the optimizer could not see through, often a function on the column. I also look for a spill, a lookup count, and a join type that made sense for the estimate and not for the actual. Query Store shows that on production without my running an actual plan against the live primary at the peak. An estimated plan alone has no actual rows. It is a guess I use to see the shape, and I say it is a guess.

Say this as a tech lead

I ask for the actual numbers from Query Store. I do not approve an index from an estimated plan on an empty developer database.

Q10. Why is SELECT * a problem on IX_Orders_Tenant_Created?

Answer. The star includes RowVersion and any column we add later. Those are not in the include list, so a covering seek becomes a seek plus key lookups. If the range is large, the optimizer often scans the clustered index instead. The API also serializes columns the screen does not show, which is a payload and a privacy problem. I list Id, Number, Status, Total, CreatedUtc. That list is covered. A new column in the DTO is a deliberate include, with a write cost I mention in the review.

Say this as a tech lead

The projection is part of the index design. SELECT * makes every future column part of the hot query without a decision.

Q11. When does OR defeat a seek, and when is OR still a keyset?

Answer. OR across different columns, Status = N'Paid' OR CustomerId = @CustomerId, on an index that leads with TenantId and CreatedUtc, gives the optimizer two predicates that index cannot satisfy together. The usual result is a scan, or a seek of the whole tenant. I rewrite that as UNION ALL of two seeks when each branch has its own index. The keyset predicate is a different OR: CreatedUtc is less than the cursor, or it is equal and Id is less. Both branches are the same ordered key. It should seek IX_Orders_Tenant_Created. I still read the plan. If that OR scans, I rewrite it as UNION ALL so each branch is an obvious range.

Say this as a tech lead

I do not ban OR. I ban an OR I cannot point at on the plan. The keyset form stays only while the plan seeks.

Q12. Who should the API connect as?

Answer. A managed identity, mapped with CREATE USER FROM EXTERNAL PROVIDER, in a custom role that has DML and EXECUTE on the orders schema. Not db_owner, and not the pipeline identity that runs DDL. db_datareader plus db_datawriter is a stepping stone I can explain, and it covers every table, including ones

this API should not touch. A new table with no grant fails safe under the custom role. Humans use their own audited accounts. The connection string uses Active Directory Managed Identity. There is no SQL password to commit.

> **Say this as a tech lead**
>
> If the panel says db_owner is fine because the app migrates at startup, I split the identity and I move DDL to the pipeline. That is the answer they are scoring.

Sample queries

Each statement is the answer you would write on the board. The paragraph under it is why you think the predicate matches an index, and what you would change if the plan scans. Parameters are typed to the columns: int for TenantId, datetime2 for instants, nvarchar for Number.

Orders for one tenant in a date range

Question. Return one tenant's orders created in a half-open range, newest first, with the columns the list screen shows.

```
SELECT Id, Number, Status, Total, CreatedUtc
FROM orders.Orders
WHERE TenantId = @TenantId
  AND CreatedUtc >= @FromUtc
  AND CreatedUtc <  @ToUtc
ORDER BY CreatedUtc DESC, Id DESC;
```

TenantId is an equality on the leading key of IX_Orders_Tenant_Created. CreatedUtc is a range on the next key, and the ORDER BY matches the index direction, so the range can be read in order without a sort. Number, Status, and Total are included, and Id is the clustering key already stored in the leaf, so the select list is covered. @FromUtc inclusive and @ToUtc exclusive keeps a row that falls on the boundary in exactly one page of a report.

The plan scans if you write YEAR(CreatedUtc) or CONVERT(date, CreatedUtc) on the column, if you omit TenantId and filter only by time, or if @TenantId arrives as nvarchar and you get a conversion you did not intend on a different column. SELECT * can turn the seek into a clustered scan once lookups dominate. A range that truly covers most of a huge tenant is a long read of that index leaf. That is a large seek range, and you say so, then you cap it or you summarize.

Top customers by total

Question. For one tenant and one date range, return the twenty customers with the highest sum of order totals.

```
SELECT TOP (20)
    c.CustomerId,
    c.DisplayName,
    SUM(o.Total) AS TotalSpend,
    COUNT_BIG(*) AS OrderCount
```

```
FROM orders.Orders AS o
INNER JOIN orders.Customers AS c
    ON c.CustomerId = o.CustomerId
   AND c.TenantId = o.TenantId
WHERE o.TenantId = @TenantId
  AND o.CreatedUtc >= @FromUtc
  AND o.CreatedUtc <  @ToUtc
GROUP BY c.CustomerId, c.DisplayName
ORDER BY TotalSpend DESC;
```

The filter is the same seek as the date-range list. Total and CustomerId are included, so the aggregate does not key-lookup the order. The join to Customers is a seek on the customer primary key, and the tenant predicate on the join refuses a customer row that does not belong to the order's tenant. GROUP BY will hash or sort the qualifying orders. That is expected. TOP (20) limits the output, not the amount of aggregation, unless the optimizer finds a cheaper ordered path. Joining OrderLines into this query multiplies Total by the line count. Keep the aggregate on Orders.

It scans when the date predicate wraps CreatedUtc, when the query is asked for every tenant at once, or when an OR such as Status = N'Paid' OR CustomerId = @CustomerId is added on columns the index cannot serve together. A missing include of Total still seeks and then looks up. That is the plan you fix by INCLUDE, after you accept the wider leaf.

Find duplicate idempotency use

Question. Show whether an idempotency key was stored twice, and show keys that clients reused with a different body.

```
-- Must return zero rows. The primary key is (TenantId, KeyValue).
-- A row here means a load path bypassed the constraint.
SELECT TenantId, KeyValue, COUNT(*) AS Uses
FROM orders.IdempotencyKeys
GROUP BY TenantId, KeyValue
HAVING COUNT(*) > 1;

-- Same key, different request hash. The API inserts this row on 409
-- and does not insert a second order.
SELECT TenantId, KeyValue, COUNT(*) AS Conflicts, MAX(RejectedUtc) AS LastSeen
FROM orders.IdempotencyRejections
GROUP BY TenantId, KeyValue
HAVING COUNT(*) >= 1;
```

The first statement is the audit of the guarantee. The clustered primary key is the index, and a correct table cannot contain a duplicate, so the having-count query is empty. You still run it after a bulk import or a replica you built without the key. The second statement uses IX_IdempotencyRejections_Key, whose leading columns are TenantId and KeyValue, which matches the group. Those rows are the business duplicates the primary key refused: same tenant, same key, different hash.

You get a scan of IdempotencyKeys if you drop the primary key and also have no index on (TenantId, KeyValue), or if you wrap KeyValue in a function. An empty rejection table is not proof that nobody retried. It is proof you did not log the 409. The order table's unique (TenantId, Number) is the other duplicate to ask

about, with the same GROUP BY shape, when the idempotency key was fine and the business number was not.

A page of orders with keyset pagination

Question. Return the next page after a cursor, without OFFSET. The first page uses a cursor later than any real row, or omits the cursor predicate.

```
SELECT TOP (@Take) Id, Number, Status, Total, CreatedUtc
FROM orders.Orders
WHERE TenantId = @TenantId
  AND (
        CreatedUtc < @CursorUtc
        OR (CreatedUtc = @CursorUtc AND Id < @CursorId)
      )
ORDER BY CreatedUtc DESC, Id DESC;
```

This is the deep page. @Take is capped in the API, for example at 100, and you say the cap is a product choice. The predicate continues the index order: everything strictly older than the cursor instant, plus the tie on Id at that same instant. IX_Orders_Tenant_Created leads with TenantId, then CreatedUtc DESC, then Id DESC, and it includes Number, Status, and Total. The page should be a seek and a short ordered range. The cursor values are the last row the client already showed. A new insert newer than the cursor does not shift this page.

OFFSET @Skip ROWS FETCH NEXT @Take ROWS ONLY is the form you show for a small admin screen and then refuse for a deep page. SQL reads and discards the skipped rows, and inserts at the top move the page. The keyset OR scans if the plan cannot see it as one range. The repair is UNION ALL: one branch for CreatedUtc < @CursorUtc, one branch for the equal instant and Id < @CursorId, each able to seek, with TOP applied carefully so you do not read the entire history. A cursor that omits Id repeats or skips rows that share CreatedUtc. An uncapped @Take is how one tenant asks for a million rows.

```
SELECT TOP (@Take) Id, Number, Status, Total, CreatedUtc
FROM (
    SELECT Id, Number, Status, Total, CreatedUtc
    FROM orders.Orders
    WHERE TenantId = @TenantId
      AND CreatedUtc < @CursorUtc
    UNION ALL
    SELECT Id, Number, Status, Total, CreatedUtc
    FROM orders.Orders
    WHERE TenantId = @TenantId
      AND CreatedUtc = @CursorUtc
      AND Id < @CursorId
) AS page
ORDER BY CreatedUtc DESC, Id DESC;
```

A tenant-safe update

Question. Mark an order paid only if it belongs to this tenant and the caller still holds the RowVersion they read.

```
UPDATE orders.Orders
SET Status = @NewStatus
WHERE TenantId = @TenantId
  AND Id = @OrderId
  AND RowVersion = @RowVersion;

-- @@ROWCOUNT = 1: updated.
-- @@ROWCOUNT = 0: read again by TenantId and Id.
--   no row  -> 404, outside this tenant or unknown id
--   a row   -> 409, someone else wrote it
```

@TenantId comes from the token, the same way the API section requires. @OrderId comes from the route. @RowVersion is binary(8), the value the client read. The clustered primary key seeks Id. TenantId and RowVersion are residual predicates on that one row. The seek stays efficient. The tenant predicate is what makes a guessed id update zero rows. Leaving it off is fast and wrong.

The statement scans if you update by a function of Number, or by a predicate with no selective key, such as Status = @Old across the table. Two writers who both read the total and both set a new total will lose one update even under row versioning. The RowVersion check is the lost-update control. A set-based alternative, Total = Total + @delta, is safer when the business rule is an increment. Either way the tenant predicate stays.

One join instead of N+1

Question. Load one order and its lines in a single statement. The N+1 shape is a query for the order, then a query per line, or a list call followed by a line call per order from Angular.

```
SELECT o.Id, o.Number, o.Total, l.Sku, l.Quantity, l.UnitPrice
FROM orders.Orders AS o
INNER JOIN orders.OrderLines AS l
    ON l.OrderId = o.Id
   AND l.TenantId = o.TenantId
WHERE o.TenantId = @TenantId
  AND o.Id = @OrderId;
```

Orders seeks the clustered primary key on Id and checks TenantId. OrderLines seeks IX_OrderLines_Order on (TenantId, OrderId) and returns Sku, Quantity, and UnitPrice from the include list, so the line side is covered. One round trip, one tenant predicate on both sides. If the screen needs twenty orders, filter those ids in one statement, still with TenantId, and group the lines in the API. EF's version of the same idea is a projection or Include with AsSplitQuery when several collections would multiply rows. Split query is two round trips on purpose, still constant, not one per line.

The line side scans if the only index is the clustered OrderLineId and you filter by OrderId, once the table is large. The order side scans if you look the order up by a non-SARGable expression on Number. Joining lines when you only wanted order totals, as in the top-customers query, multiplies the sum. That is a different bug. A loop in C# that runs this statement once per order id is still N+1 at the API boundary, just with a nicer inner statement.

A deadlock, and the lock order that removes it

Question. Two sessions update an order and its lines. Show the cycle, then show the rule you enforce.

```
-- Session A. Parent, then child.
BEGIN TRAN;
UPDATE orders.Orders
SET Status = N'Paid'
WHERE Id = 10 AND TenantId = 1;

UPDATE orders.OrderLines
SET Quantity = Quantity
WHERE OrderId = 10 AND TenantId = 1;
COMMIT;

-- Session B. Child, then parent. Cycle. One session dies with 1205.
BEGIN TRAN;
UPDATE orders.OrderLines
SET Quantity = Quantity + 1
WHERE OrderId = 10 AND TenantId = 1;

UPDATE orders.Orders
SET Status = N'Packed'
WHERE Id = 10 AND TenantId = 1;
COMMIT;
```

Session A holds the order row and wants the lines. Session B holds the lines and wants the order. The fix is a rule every transaction follows: touch Orders first, then OrderLines, then anything else you must name. Keep the transaction free of HTTP. The point updates seek Id and (TenantId, OrderId), so they lock the rows they mean rather than a scanned range. The same cycle happens on one table when session A updates id 10 then id 12 and session B updates id 12 then id 10. One set-based update, or a locked order by Id, removes that cycle too.

```
-- Both sessions, same order. The HTTP call to take payment stays outside.
BEGIN TRAN;
UPDATE orders.Orders
SET Status = N'Paid'
WHERE Id = @OrderId AND TenantId = @TenantId;

UPDATE orders.OrderLines
SET Quantity = Quantity
WHERE OrderId = @OrderId AND TenantId = @TenantId;
COMMIT;
```

A missing index on the line filter turns the second update into a scan that locks many tenants' lines and creates cycles you did not draw. Retrying 1205 is allowed when the unique idempotency key makes the whole transaction safe to run again. Retrying a payment insert that has no unique key charges twice. The graph is the evidence. The metric on 1205 is how you notice the retry is hiding a design problem.

Interview trap

Offering NOLOCK on the read side of this bug. The deadlock is a write cycle. A dirty read does not give the two updates a single lock order. Offering a bigger App Service plan does not either. The plan runs the same two statements on more connections.

Say this

I write the tenant predicate on every statement, I keep the column bare so the index can seek, and I touch tables in one order. The unique key and the RowVersion token are what C# cannot guarantee across two instances.

A short drill

1. On paper, write the date-range select and name the index key columns in order.
2. Rewrite YEAR(CreatedUtc) = 2026 so it can seek, and say what SELECT * does to that plan.
3. Write the tenant-safe update. Say what @@ROWCOUNT of 0 means the second time you read the row.
4. Draw two sessions deadlocking on Orders and OrderLines, then write the one order both will use.